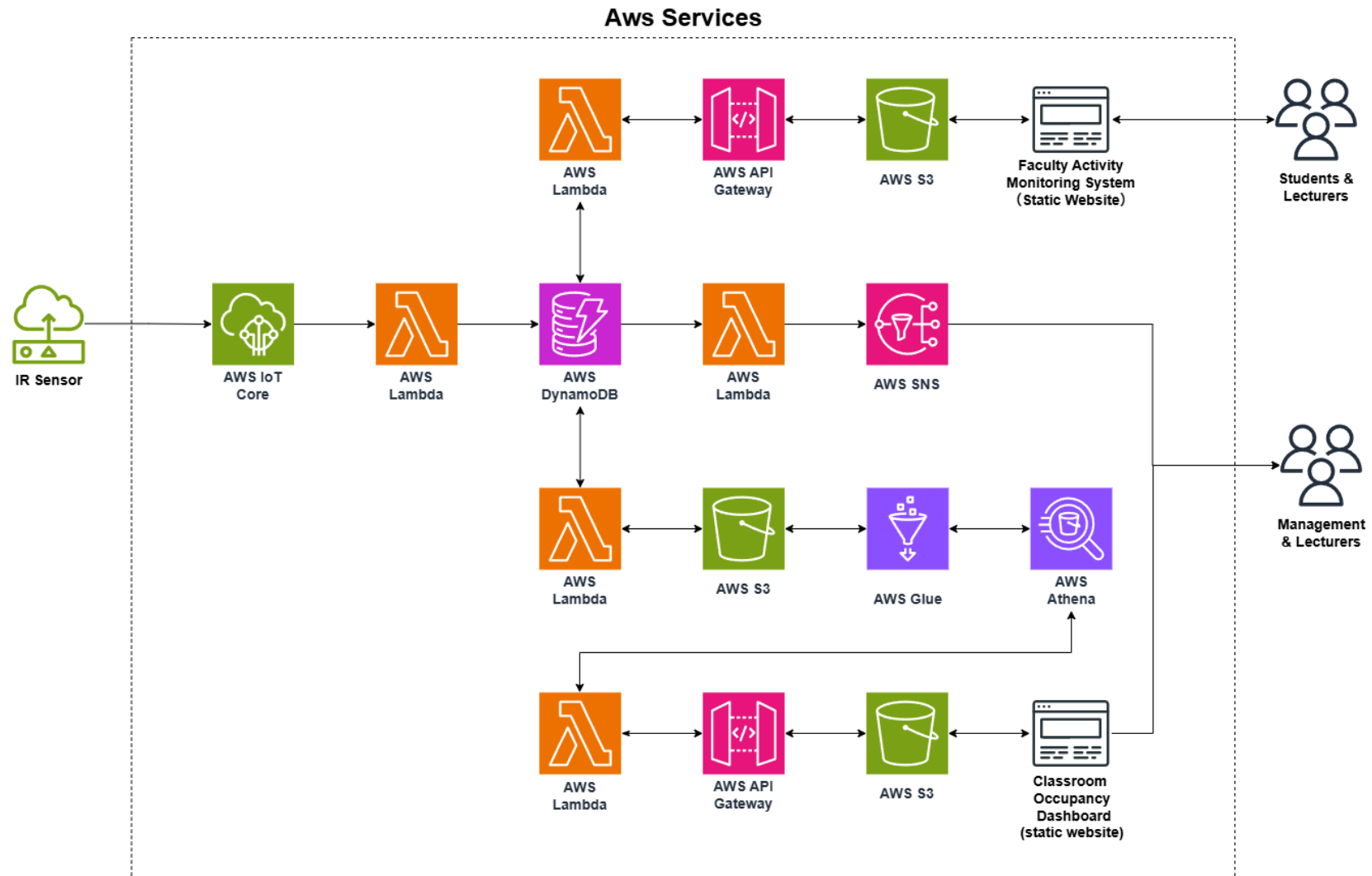


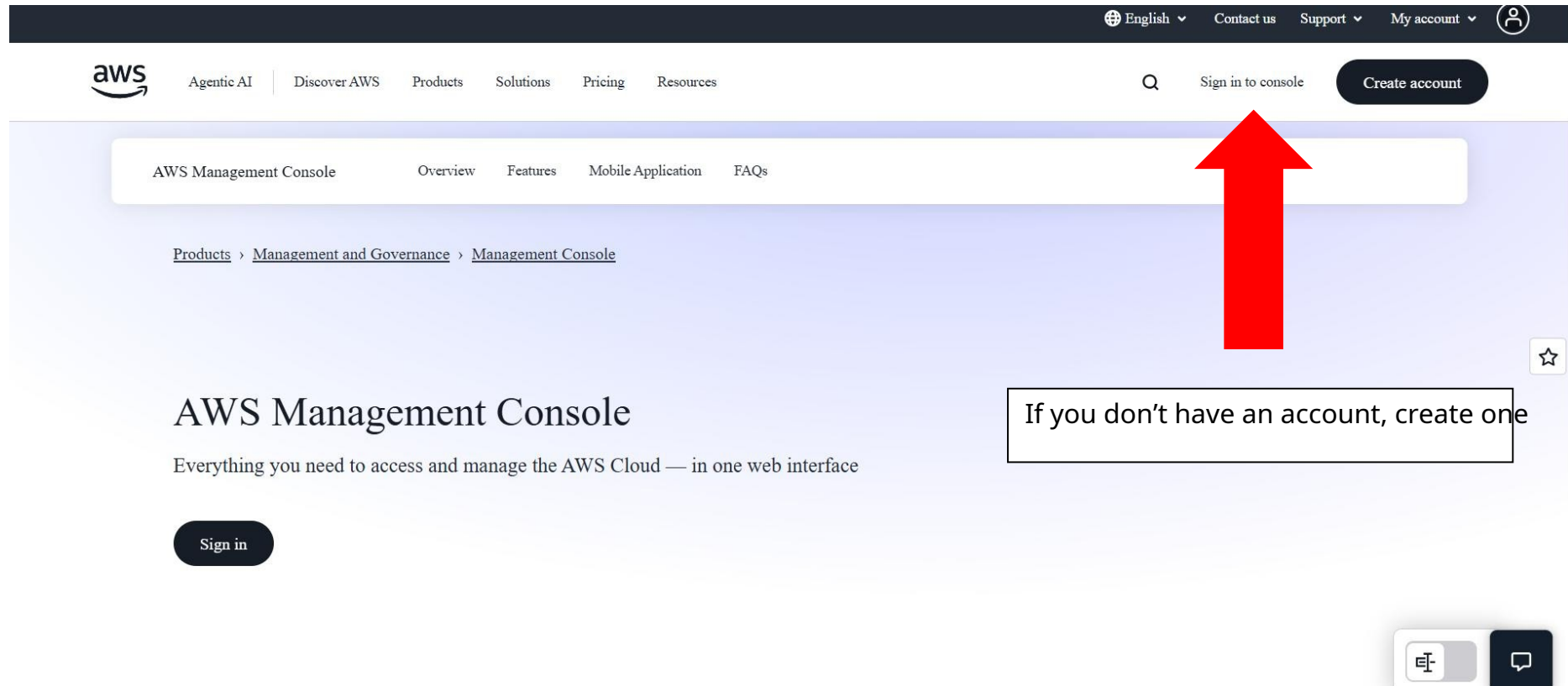
Faculty Activity Monitoring & Space Utilisation Analytics



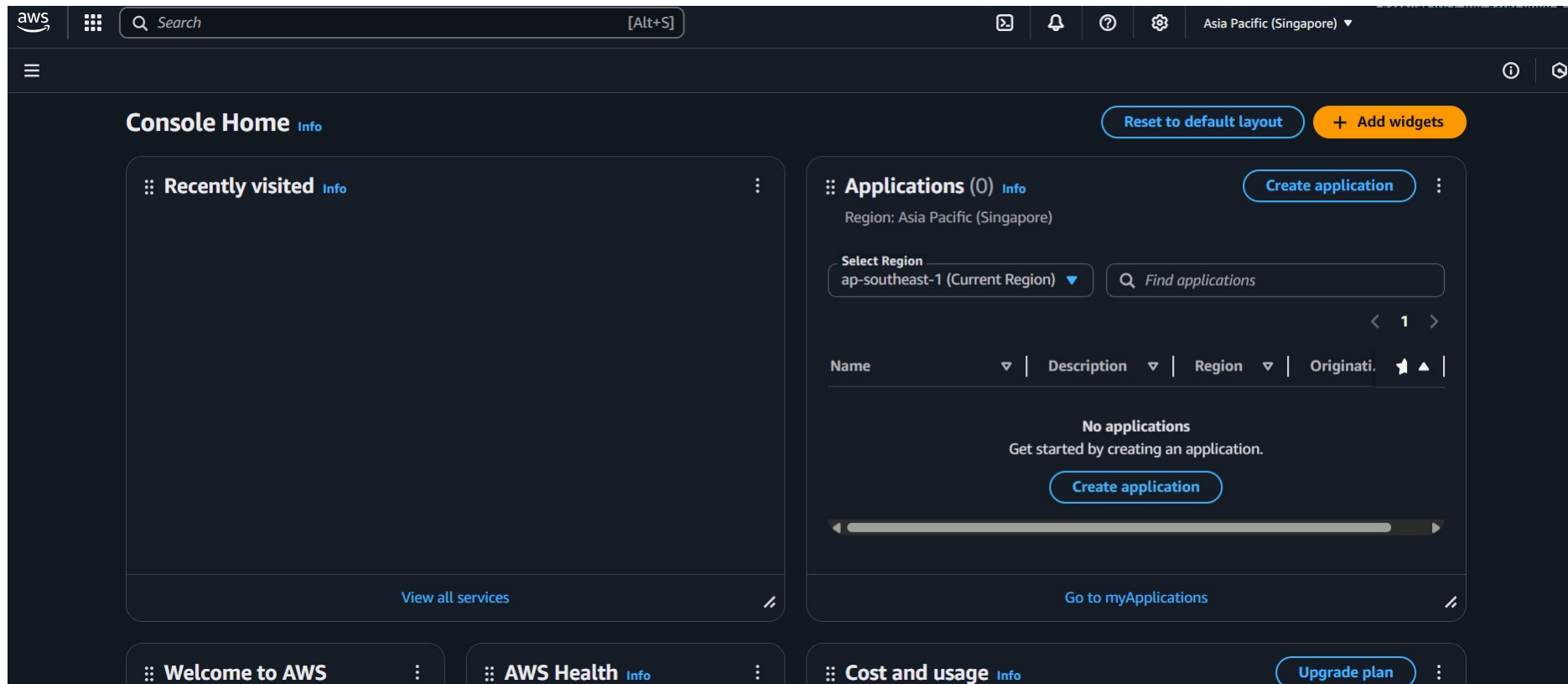
Amazon Platform Configuration

AWS Console

Sign in to the AWS Console



AWS Console Home



Search services at the *Find Services* search bar

The screenshot shows the AWS Find Services interface. At the top, the AWS logo is on the left, and a search bar contains the text "iot core". To the right of the search bar are icons for a terminal and a notification bell. Below the search bar, a left-hand navigation menu lists various categories: Services (highlighted with a blue bar), Features, Resources (with a "New" tag), Documentation, Knowledge articles, Marketplace, Blog posts, Tutorials, and Events. The main content area is divided into two sections. The "Services" section, titled "Services" with a "Show more" link, lists three items: "IoT Core" (Connect Devices to the Cloud), "AWS IoT Core for LoRaWAN" (Connect, manage, and secure LoRaWAN devices at scale), and "Amazon Fraud Detector" (Detect more online fraud faster using machine learning). Each item has a star icon for favoriting. The "Features" section, titled "Features" with a "Show more" link, lists three items: "Device Advisor" (IoT Core feature), "Device logs" (IoT Core feature), and "Secure tunneling". At the bottom left, a feedback prompt asks "Were these results helpful?" with "Yes" and "No" buttons.

aws

Q iot core

Services

Services

Features

Resources **New**

Documentation

Knowledge articles

Marketplace

Blog posts

Tutorials

Events

IoT Core
Connect Devices to the Cloud

AWS IoT Core for LoRaWAN
Connect, manage, and secure LoRaWAN devices at scale

Amazon Fraud Detector
Detect more online fraud faster using machine learning

Features

Device Advisor
IoT Core feature

Device logs
IoT Core feature

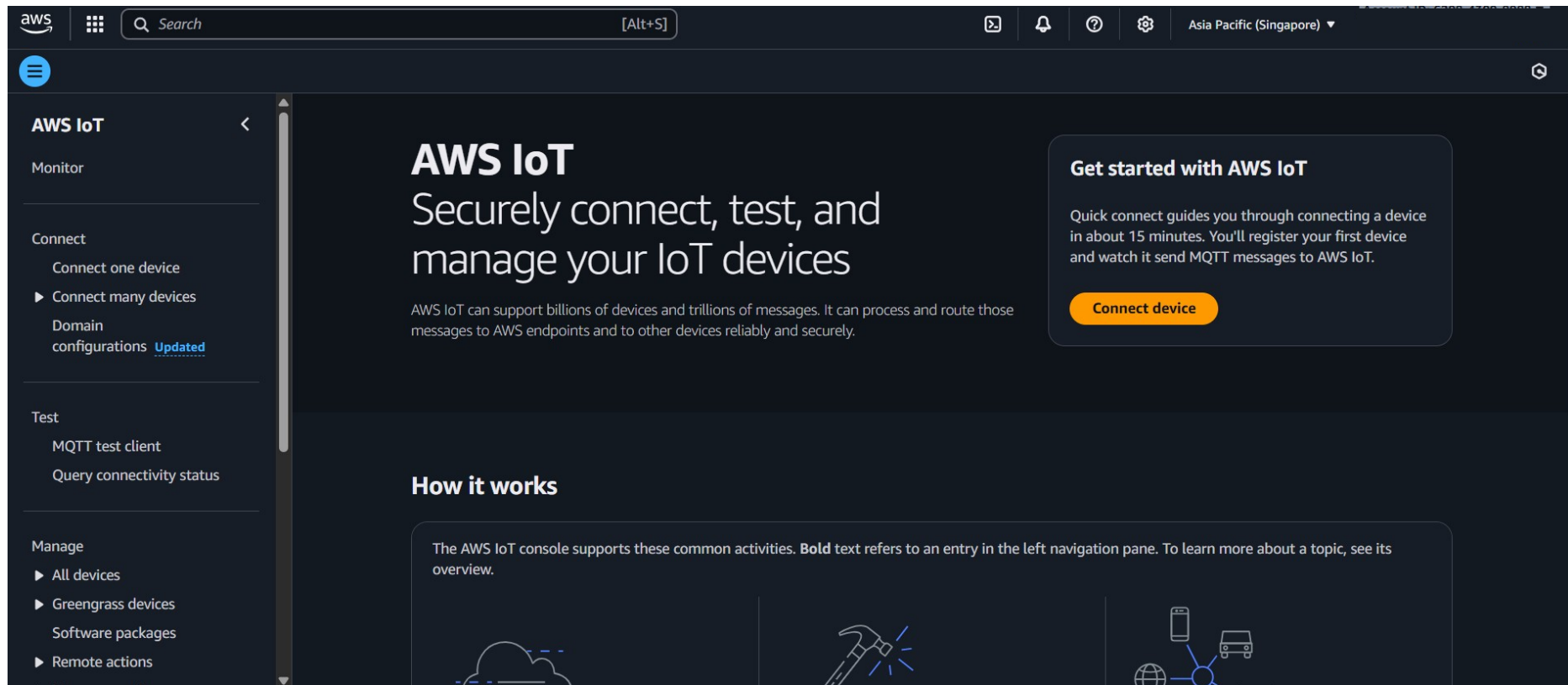
Secure tunneling

Were these results helpful?

Yes No

IoT Core

IoT Core Home Page



The screenshot displays the AWS IoT Core console interface. At the top, the AWS logo is on the left, followed by a search bar and a navigation menu with icons for help, notifications, and settings. The region is set to 'Asia Pacific (Singapore)'. The left-hand navigation pane is titled 'AWS IoT' and includes sections for 'Monitor', 'Connect' (with sub-items 'Connect one device', 'Connect many devices', and 'Domain configurations Updated'), 'Test' (with 'MQTT test client' and 'Query connectivity status'), and 'Manage' (with 'All devices', 'Greengrass devices', 'Software packages', and 'Remote actions'). The main content area features a large header with the text 'AWS IoT Securely connect, test, and manage your IoT devices' and a sub-header 'AWS IoT can support billions of devices and trillions of messages. It can process and route those messages to AWS endpoints and to other devices reliably and securely.' To the right of this header is a 'Get started with AWS IoT' section with a 'Connect device' button. Below the header is a 'How it works' section with a text box stating: 'The AWS IoT console supports these common activities. **Bold** text refers to an entry in the left navigation pane. To learn more about a topic, see its overview.' At the bottom of the 'How it works' section, there are three icons: a cloud, a hammer, and a car connected to a network.

aws [Search] [Alt+S] Asia Pacific (Singapore)

AWS IoT

Securely connect, test, and manage your IoT devices

AWS IoT can support billions of devices and trillions of messages. It can process and route those messages to AWS endpoints and to other devices reliably and securely.

Get started with AWS IoT

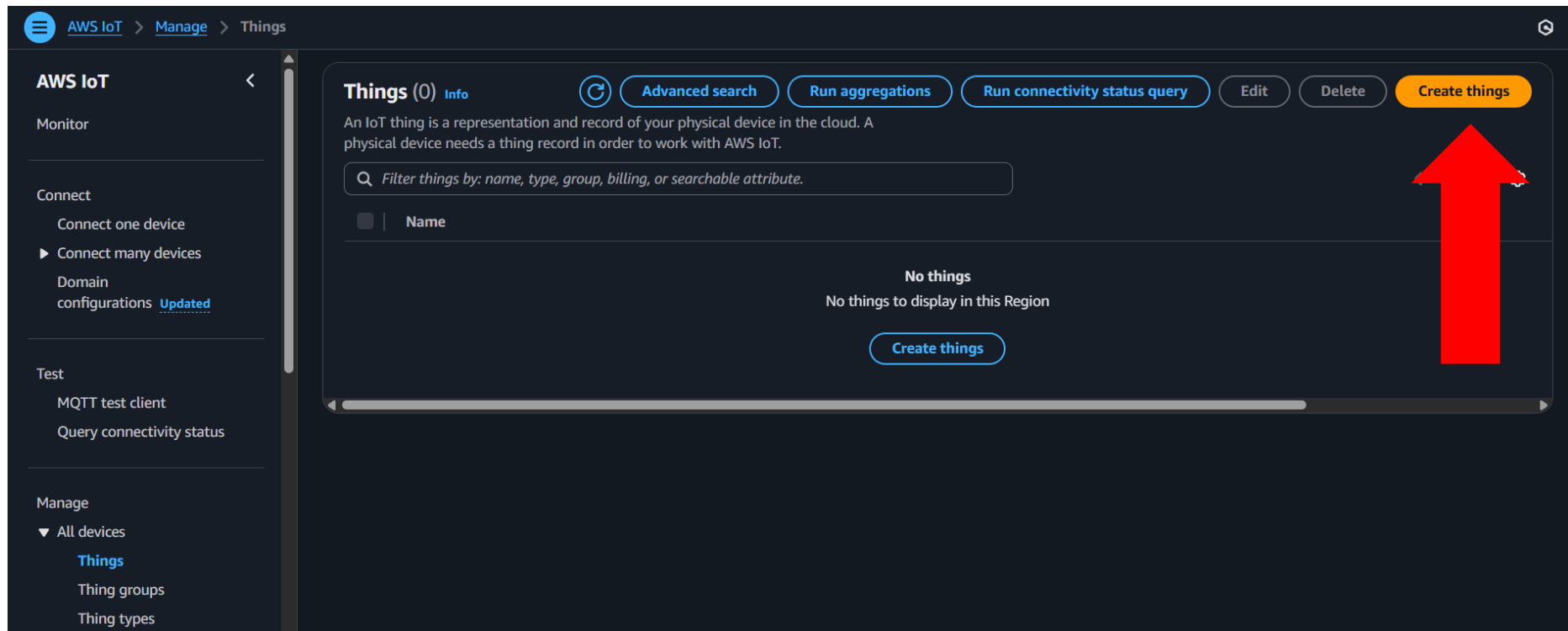
Quick connect guides you through connecting a device in about 15 minutes. You'll register your first device and watch it send MQTT messages to AWS IoT.

Connect device

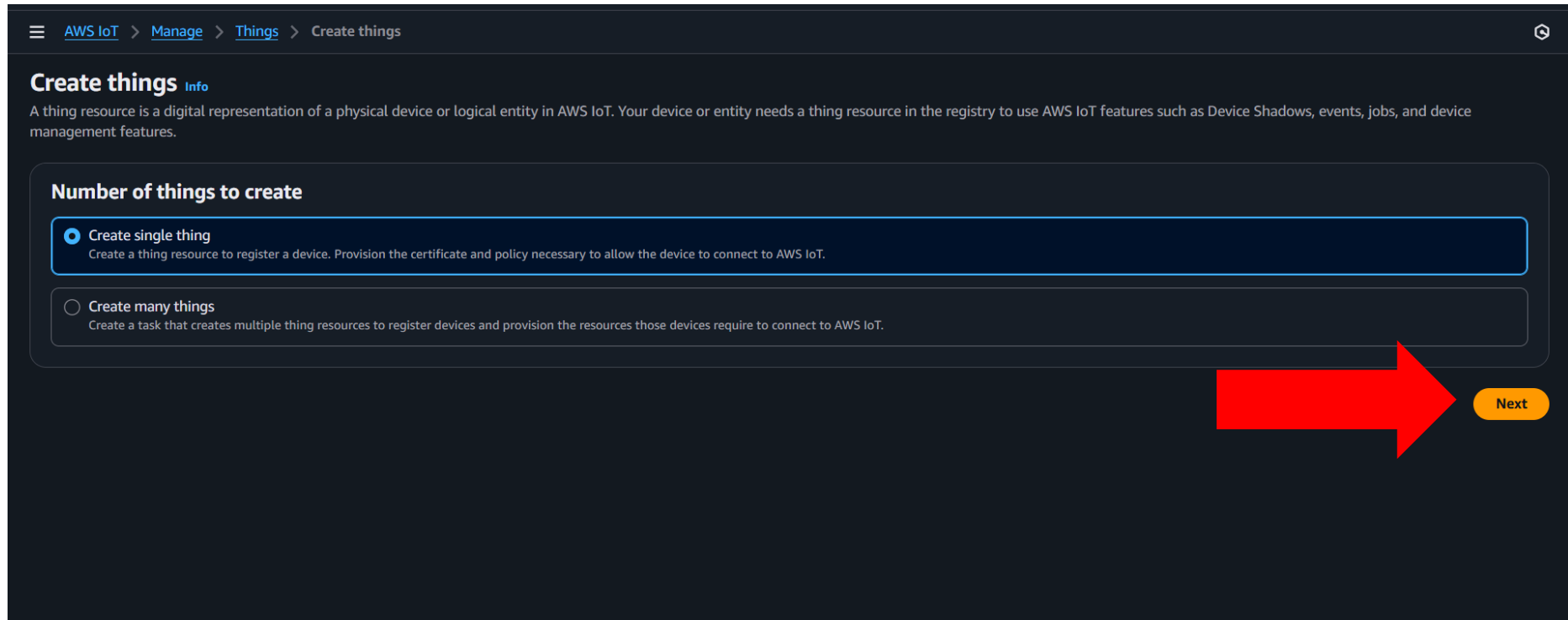
How it works

The AWS IoT console supports these common activities. **Bold** text refers to an entry in the left navigation pane. To learn more about a topic, see its overview.

Navigate to *Things* in *All devices* under *Manage* and Create things



Choose the number of items to create, and click Next



☰ [AWS IoT](#) > [Manage](#) > [Things](#) > Create things ⌕

Create things [Info](#)

A thing resource is a digital representation of a physical device or logical entity in AWS IoT. Your device or entity needs a thing resource in the registry to use AWS IoT features such as Device Shadows, events, jobs, and device management features.

Number of things to create

- ☒ **Create single thing**
Create a thing resource to register a device. Provision the certificate and policy necessary to allow the device to connect to AWS IoT.
- ☐ **Create many things**
Create a task that creates multiple thing resources to register devices and provision the resources those devices require to connect to AWS IoT.

Next

Input thing's name and click Next

Step 1

☒ Specify thing properties

☐ Step 2 - optional
Configure device certificate

☐ Step 3 - optional
Attach policies to certificate

Specify thing properties [Info](#)

A thing resource is a digital representation of a physical device or logical entity in AWS IoT. Your device or entity needs a thing resource in the registry to use AWS IoT features such as Device Shadows, events, jobs, and device management features.

Thing properties [Info](#)

Thing name

Enter a unique name containing only: letters, numbers, hyphens, colons, or underscores. A thing name can't contain any spaces.

Additional configurations

You can use these configurations to add detail that can help you to organize, manage, and search your things.

- ▶ Thing type - optional
- ▶ Searchable thing attributes - optional
- ▶ Thing groups - optional
- ▶ Billing group - optional
- ▶ Packages and versions - optional

Device Shadow [Info](#)

Device Shadows allow connected devices to sync states with AWS. You can also get, update, or delete the state information of this thing's shadow using either HTTPs or MQTT topics.

☒ No shadow

☐ Named shadow
Create multiple shadows with different names to manage access to properties, and logically group your devices properties.

☐ Unnamed shadow (classic)
A thing can have only one unnamed shadow.

[Cancel](#) [Next](#)

Click Next to continue

The screenshot shows the 'Configure device certificate' step in the AWS IoT console. On the left, a progress bar indicates four steps: 'Specify thing properties' (Step 1), 'Configure device certificate' (Step 2, currently active), 'Attach policies to certificate' (Step 3 - optional), and 'Attach policies to certificate' (Step 4 - optional). The main content area is titled 'Configure device certificate - optional' with an 'info' icon. Below the title, a paragraph explains that a device requires a certificate to connect to AWS IoT and offers two options: registering a certificate now or creating and registering one later. The 'Device certificate' section contains four radio button options: 'Auto-generate a new certificate (recommended)' (selected), 'Use my certificate', 'Upload CSR', and 'Skip creating a certificate at this time'. Each option has a brief description. At the bottom right, there are three buttons: 'Cancel', 'Previous', and 'Next'. A large red arrow points directly to the 'Next' button.

Step 1
● Specify thing properties

Step 2 - optional
● **Configure device certificate**

Step 3 - optional
○ Attach policies to certificate

Configure device certificate - optional [info](#)

A device requires a certificate to connect to AWS IoT. You can choose how to register a certificate for your device now, or you can create and register a certificate for your device later. Your device won't be able to connect to AWS IoT until it has an active certificate with an appropriate policy.

Device certificate

- ☒ **Auto-generate a new certificate (recommended)**
Generate a certificate, public key, and private key using AWS IoT's certificate authority.
- ☐ **Use my certificate**
Use a certificate signed by your own certificate authority.
- ☐ **Upload CSR**
Register your CA and use your own certificates on one or many devices.
- ☐ **Skip creating a certificate at this time**
You can create a certificate for this thing and attach a policy to the certificate at a later time.

[Cancel](#) [Previous](#) [Next](#)

Create policy

The screenshot shows the AWS IoT console interface. At the top, the navigation bar includes the AWS logo, a search bar, and the account ID: 6298-4300-8988. The breadcrumb trail indicates the current path: AWS IoT > Manage > Things > Create things > Create single thing.

On the left, a sidebar shows the progress of the setup steps:

- Step 1: Specify thing properties
- Step 2 - optional: Configure device certificate
- Step 3 - optional: **Attach policies to certificate** (current step)

The main content area is titled "Attach policies to certificate - optional" with an "Info" link. Below the title, a message states: "AWS IoT policies grant or deny access to AWS IoT resources. Attaching policies to the device certificate applies this access to the device."

The "Policies (0)" section includes a search bar labeled "Filter policies" and a table header with a checkbox and the column "Name". The table is currently empty, displaying the message: "No policies. No policies could be found in ap-southeast-1."

In the top right corner of the policies section, there is a circular refresh icon and a "Create policy" button with an external link icon. A large red arrow points directly to this "Create policy" button.

At the bottom right of the console, there are three buttons: "Cancel", "Previous", and "Create thing".

Input policy's details

Create policy [Info](#)

AWS IoT Core policies allow you to manage access to the AWS IoT Core data plane operations.

Policy properties

AWS IoT Core supports named policies so that many identities can reference the same policy document.

Policy name

A policy name is an alphanumeric string that can also contain period (.), comma (,), hyphen (-), underscore (_), plus sign (+), equal sign (=), and at sign (@) characters, but no spaces.

► **Tags - optional**

Policy statements | **Policy examples**

Policy document [Info](#)

An AWS IoT policy contains one or more policy statements. Each policy statement contains actions, resources, and an effect that grants or denies the actions by the resources.

Policy effect **Policy action** **Policy resource**

[Add new statement](#) [Remove](#)

[Cancel](#) [Create](#)

1) Enter the policy name
2) Set '*' in both the *Policy action* and *Policy resource* fields to include all IoT default policies

Go back to the Create thing tab and attach the policy

Step 1
Specify thing properties

Step 2 - optional
Configure device certificate

Step 3 - optional
Attach policies to certificate

Attach policies to certificate - optional [Info](#)

AWS IoT policies grant or deny access to AWS IoT resources. Attaching policies to the device certificate applies this access to the device.

Policies (1/1)

Select up to 10 policies to attach to this certificate.

☒	Name
☒	IRsensor_policy

Cancel Previous **Create thing**

Download 4 certificates

×

Download certificates and keys

Download certificate and key files to install on your device so that it can connect to AWS.

Download certificates and keys
Download certificate and key files to install on your device so that it can connect to AWS.

Device certificate
254efabe9b4...te.pem.crt

Deactivate certificate

Download

Key files
The key files are unique to this certificate and can't be downloaded after you leave this page. Download them now and save them in a secure place.

⚠ This is the only time you can download the key files for this certificate.

Public key file
254efabe9b480348066605e...e4c9835-public.pem.key

Download

Private key file
254efabe9b480348066605e...4c9835-private.pem.key

Download

Root CA certificates
Download the root CA certificate file that corresponds to the type of data endpoint and cipher suite you're using. You can also download the root CA certificates later.

Amazon trust services endpoint
RSA 2048 bit key: Amazon Root CA 1

Download

Download all

Done



Simulation Installation

Node.js Installation

Install Node.js

Download Node.js®

Get Node.js® v22.19.0 (LTS) for Windows using Docker with npm

Info Want new features sooner? Get the latest Node.js... instead and try the latest improvements!

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started
3
4 # Pull the Node.js Docker image:
5 docker pull node:22-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:22-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v22.19.0".
12
13 # Verify npm version:
14 npm -v # Should print "10.9.3".
```

PowerShell Copy to clipboard

Docker is a containerization platform. If you encounter any issues please visit [Docker's website](#)

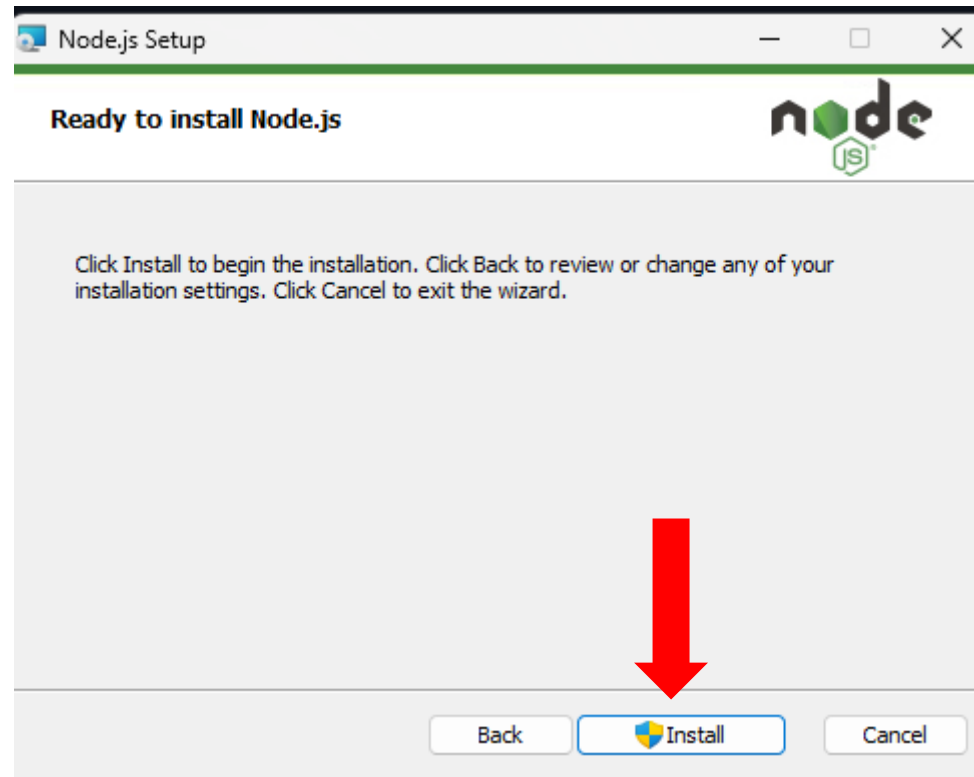
Or get a prebuilt Node.js® for Windows running a x64 architecture.

Windows Installer (.msi) Standalone Binary (.zip)

Choose the appropriate installer for your operating system

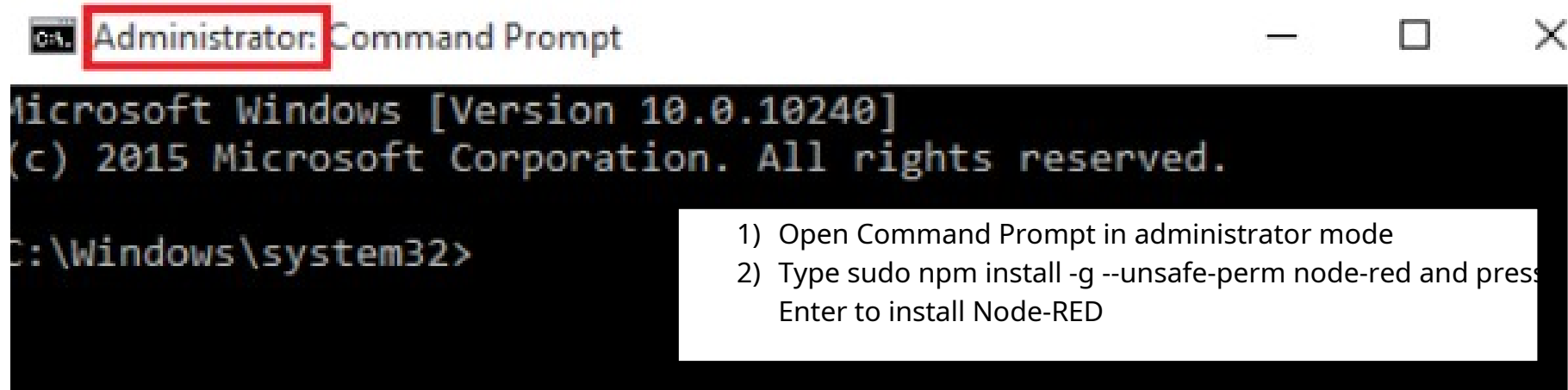
If Node.js is already installed, continue with Node-RED installation

Execute Node.js Setup



Node-RED Installation

Run command prompt



Run the Node-RED

```
C:\Users\user>node-red
31 Aug 18:01:23 - [info]
Welcome to Node-RED
=====
31 Aug 18:01:23 - [info] Node-RED version: v4.1.0
31 Aug 18:01:23 - [info] Node.js version: v22.18.0
31 Aug 18:01:23 - [info] Windows_NT 10.0.26100 x64 LE
31 Aug 18:01:23 - [info] Loading palette nodes
31 Aug 18:01:24 - [info] Dashboard version 3.6.5 started at /ui
31 Aug 18:01:24 - [info] Settings file : C:\Users\user\.node-red\settings.js
31 Aug 18:01:24 - [info] Context store : 'default' [module=memory]
31 Aug 18:01:24 - [info] User directory : \Users\user\.node-red
31 Aug 18:01:24 - [warn] Projects disabled : editorTheme.projects.enabled=false
31 Aug 18:01:24 - [info] Flows file : \Users\user\.node-red\flows.json
31 Aug 18:01:24 - [info] Server now running at http://127.0.0.1:1880/
31 Aug 18:01:24 - [warn]
```

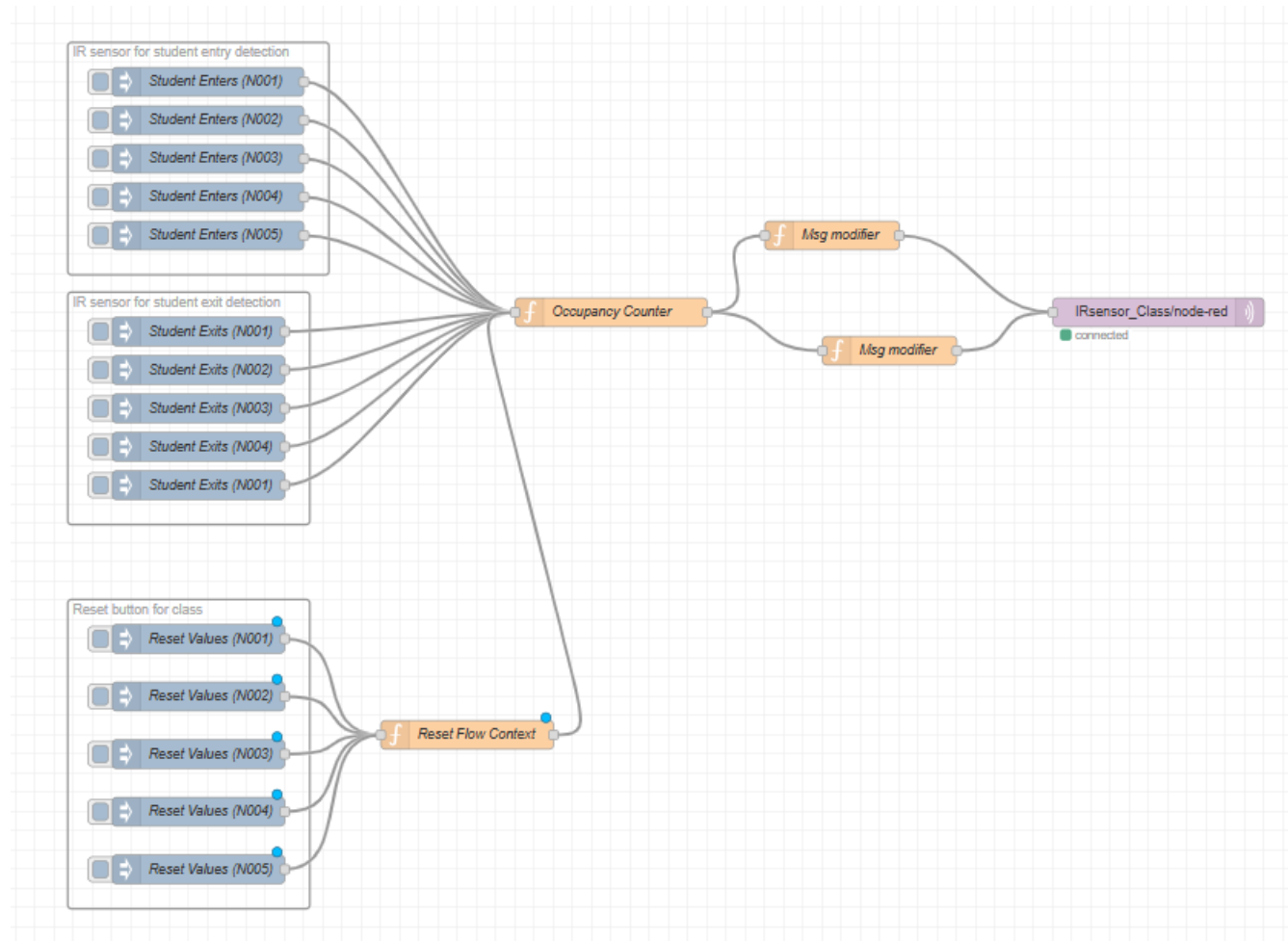
- 1) Type node-red in Command Prompt (any mode)
- 2) Ctrl + Click <http://localhost:1880> to open Node-RED in your browser

Your flow credentials file is encrypted using a system-generated key.

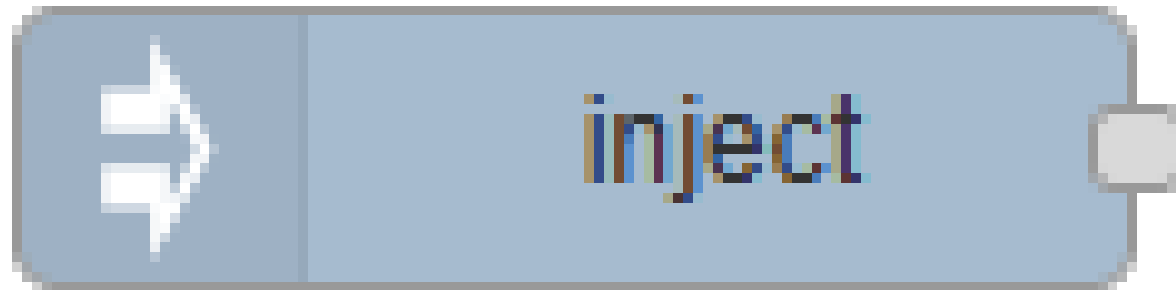
If the system-generated key is lost for any reason, your credentials file will not be recoverable, you will have to delete it and re-enter your credentials.

You should set your own key using the 'credentialSecret' option in your settings file. Node-RED will then re-encrypt your credentials file using your chosen key the next time you deploy a change.

Create the diagram flow



Drag inject node from *common* to start a flow. Double-click to configure.



Configure the IR sensor for student entry detection in Class N001

The screenshot shows a dialog box titled "Edit inject node". At the top, there are three buttons: "Delete", "Cancel", and "Done". Below these is a "Properties" section with a gear icon and three sub-icons. The "Name" field is set to "Student Enters (N001)". Below this, there are four configuration rows, each with a menu icon, a field, an equals sign, a dropdown menu, and a delete icon (X).

Field	Value
msg. payload	{ "Name": "IR sensor for entry detection (N001)" }
msg. topic	entry
msg. classId	N001
msg. noOfStudent	1

Repeat the same step for other classes

Configure the IR sensor for student exit detection in Class N001

Edit inject node

Delete Cancel Done

⚙ Properties

📁 Name Student Exits (N001)

msg. payload	=	{ "Name": "IR sensor for exit detection (N001)" }	✕
msg. topic	=	Exit	✕
msg. classId	=	N001	✕
msg. noOfStudent	=	-5	✕

Repeat the same step for other classes

Configure the Reset button for Class N001

The screenshot shows a software interface titled "Edit inject node". At the top, there are three buttons: "Delete", "Cancel", and "Done". Below these is a "Properties" section with a gear icon and three sub-icons (a gear, a document, and a camera). The "Name" field is set to "Reset Values (N001)". Below this, there are two configuration rows. The first row shows "msg. payload" on the left, followed by an equals sign, a dropdown menu with "a_z" selected, and the text "reset". The second row shows "msg. classId" on the left, followed by an equals sign, a dropdown menu with "a_z" selected, and the text "N001". At the bottom of the dialog, there is a text box containing the instruction: "Repeat the same step for other classes".

Edit inject node

Delete Cancel Done

⚙️ Properties

📁 Name Reset Values (N001)

msg. payload = a_z reset

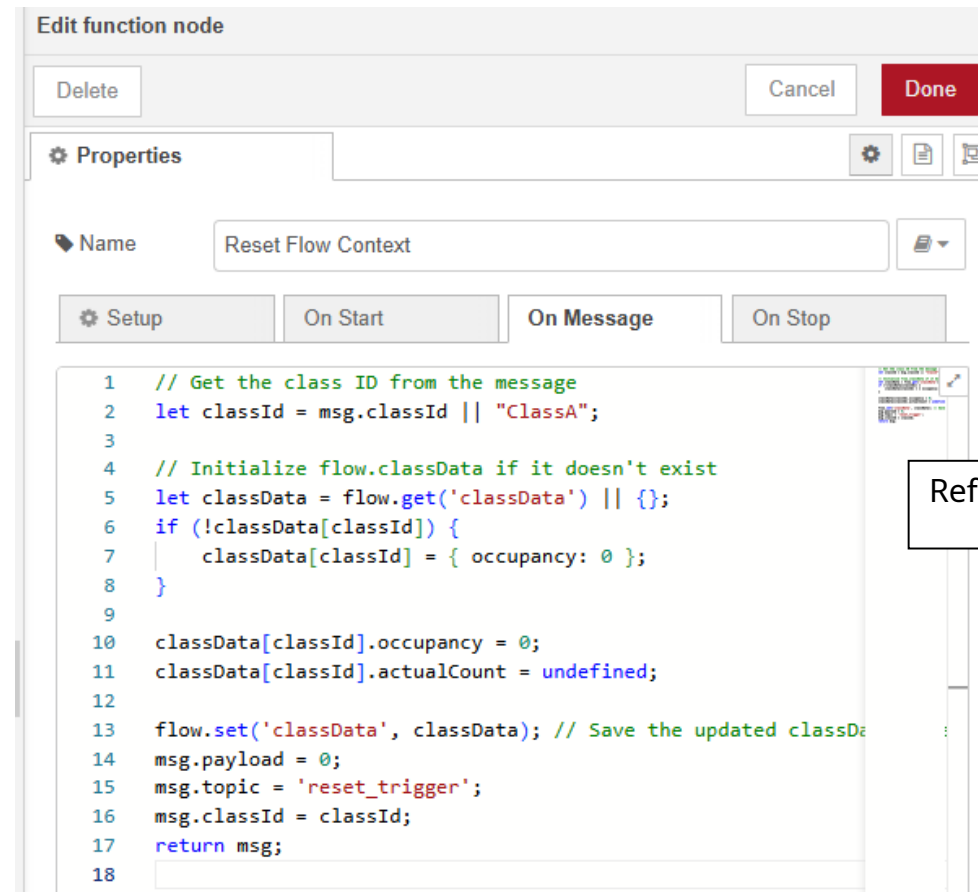
msg. classId = a_z N001

Repeat the same step for other classes

Drag function node from *function* to run custom JavaScript.
Double-click to configure.



Configure the Reset Flow Context to reset the student count in the class



Edit function node

Delete Cancel Done

Properties

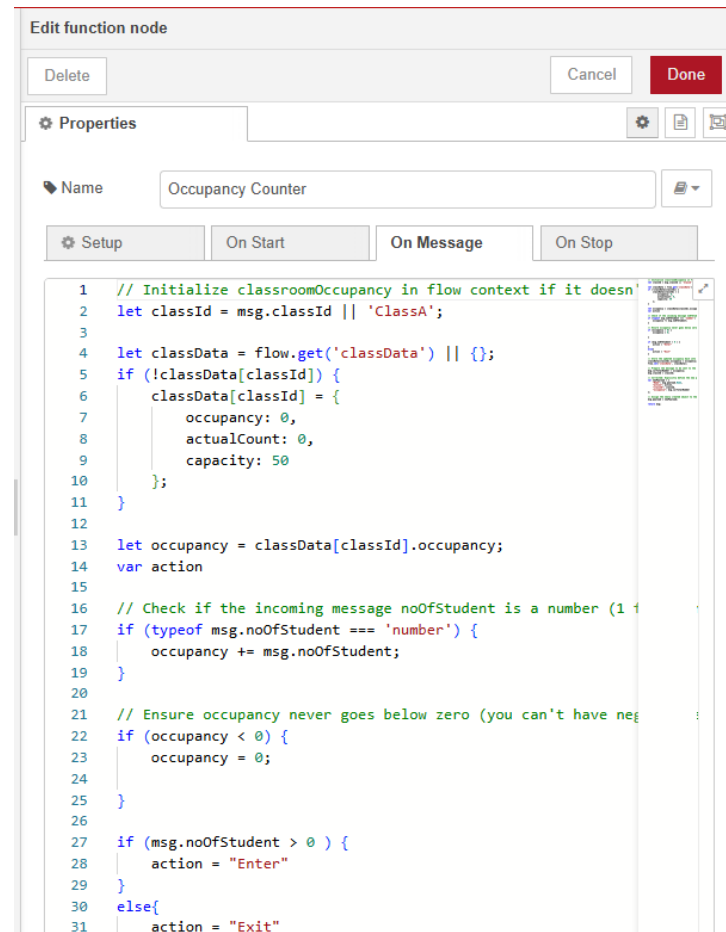
Name Reset Flow Context

Setup On Start On Message On Stop

```
1 // Get the class ID from the message
2 let classId = msg.classId || "ClassA";
3
4 // Initialize flow.classData if it doesn't exist
5 let classData = flow.get('classData') || {};
6 if (!classData[classId]) {
7     classData[classId] = { occupancy: 0 };
8 }
9
10 classData[classId].occupancy = 0;
11 classData[classId].actualCount = undefined;
12
13 flow.set('classData', classData); // Save the updated classData
14 msg.payload = 0;
15 msg.topic = 'reset_trigger';
16 msg.classId = classId;
17 return msg;
18
```

Refer to the Appendix for the code

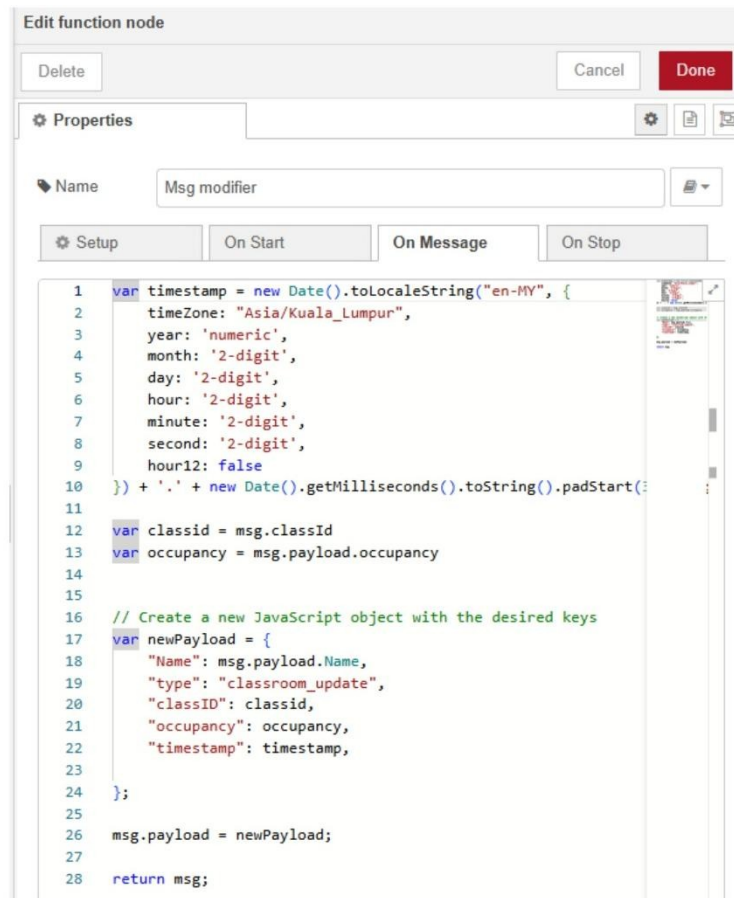
Configure the Occupancy Counter to track students entering and leaving



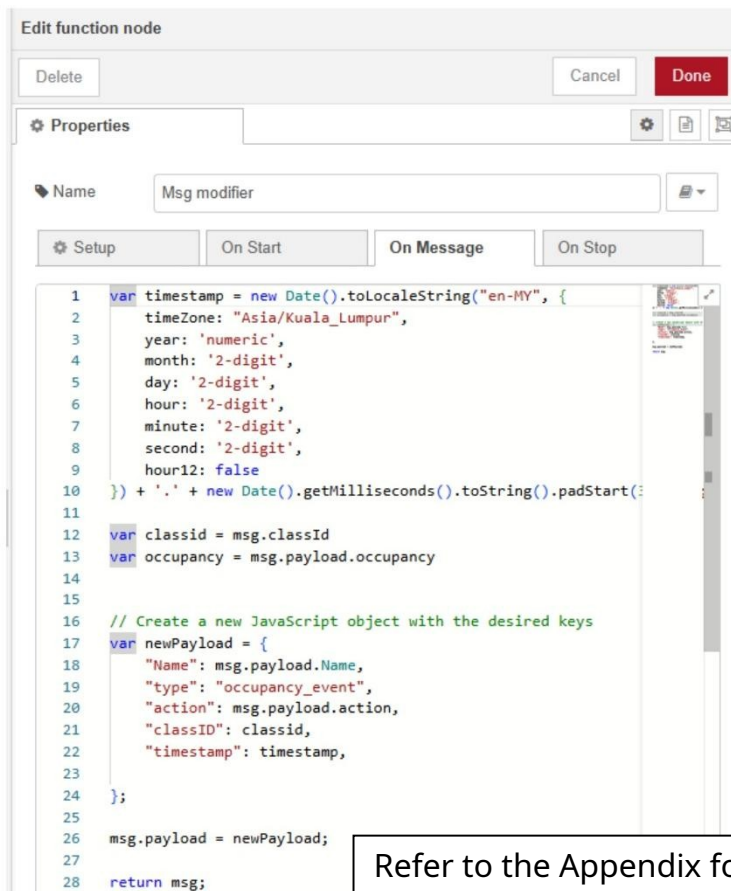
```
1 // Initialize classroomOccupancy in flow context if it doesn't exist
2 let classId = msg.classId || 'ClassA';
3
4 let classData = flow.get('classData') || {};
5 if (!classData[classId]) {
6   classData[classId] = {
7     occupancy: 0,
8     actualCount: 0,
9     capacity: 50
10  };
11 }
12
13 let occupancy = classData[classId].occupancy;
14 var action
15
16 // Check if the incoming message noOfStudent is a number (1 if enter, -1 if exit)
17 if (typeof msg.noOfStudent === 'number') {
18   occupancy += msg.noOfStudent;
19 }
20
21 // Ensure occupancy never goes below zero (you can't have negative students)
22 if (occupancy < 0) {
23   occupancy = 0;
24 }
25
26 if (msg.noOfStudent > 0) {
27   action = "Enter"
28 }
29 else{
30   action = "Exit"
```

Refer to the Appendix for the code

Configure both Msg Modifier nodes to customize messages



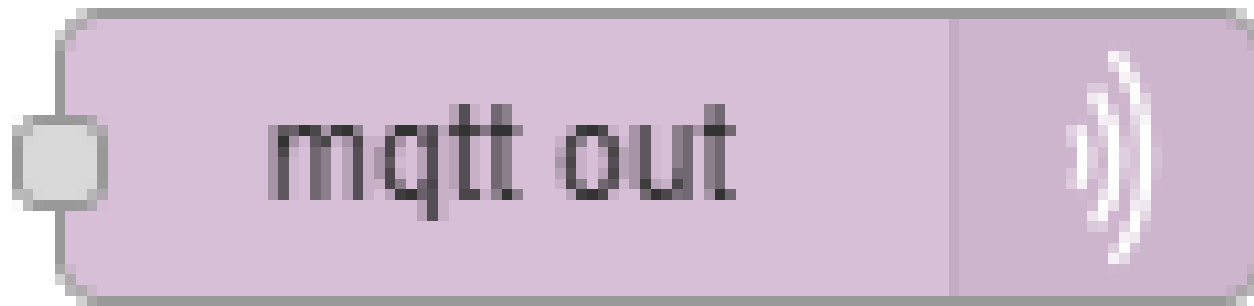
```
1 var timestamp = new Date().toLocaleString("en-MY", {
2   timeZone: "Asia/Kuala_Lumpur",
3   year: 'numeric',
4   month: '2-digit',
5   day: '2-digit',
6   hour: '2-digit',
7   minute: '2-digit',
8   second: '2-digit',
9   hour12: false
10 }) + '.' + new Date().getMilliseconds().toString().padStart(3, '0');
11
12 var classid = msg.classId
13 var occupancy = msg.payload.occupancy
14
15 // Create a new JavaScript object with the desired keys
16 var newPayload = {
17   "Name": msg.payload.Name,
18   "type": "classroom_update",
19   "classID": classid,
20   "occupancy": occupancy,
21   "timestamp": timestamp,
22 };
23
24 };
25
26 msg.payload = newPayload;
27
28 return msg;
```



```
1 var timestamp = new Date().toLocaleString("en-MY", {
2   timeZone: "Asia/Kuala_Lumpur",
3   year: 'numeric',
4   month: '2-digit',
5   day: '2-digit',
6   hour: '2-digit',
7   minute: '2-digit',
8   second: '2-digit',
9   hour12: false
10 }) + '.' + new Date().getMilliseconds().toString().padStart(3, '0');
11
12 var classid = msg.classId
13 var occupancy = msg.payload.occupancy
14
15 // Create a new JavaScript object with the desired keys
16 var newPayload = {
17   "Name": msg.payload.Name,
18   "type": "occupancy_event",
19   "action": msg.payload.action,
20   "classID": classid,
21   "timestamp": timestamp,
22 };
23
24 };
25
26 msg.payload = newPayload;
27
28 return msg;
```

Refer to the Appendix for the code

Drag mqtt out node to connect with AWS IoT Core. Double-click to configure



Input the Topic field

Edit mqtt out node

Delete Cancel Done

Properties [Settings] [File] [View]

Server none [Edit] [Add] ←

Topic IRsensor_Class/node-red ←

QoS 1 [Dropdown] Retain

Name Name

1) Enter Topic name
2) Click the Add in Server field

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

Navigate to Domain configuration under *Connect* in AWS IoT Core

The screenshot shows the AWS IoT Core console interface. On the left is a navigation sidebar with sections: **AWS IoT** (Monitor), **Connect** (Connect one device, Connect many devices, **Domain configurations Updated**), **Test** (MQTT test client, Query connectivity status), and **Manage** (All devices, Greengrass devices, Software packages, Remote actions). The main content area is titled **Domain configurations** with a subtitle: "A custom endpoint that devices use to connect to AWS IoT. Use different endpoints for different fleets to specify different authentication methods and protocols, and limit risks to a subset of devices in your fleet." Below this is a "How it works" section with three steps: 1. Select authentication types and protocols, 2. Create a new domain configuration and configure devices, and 3. Configure IoT connection policy with domain restrictions. At the bottom, a table titled "Domain configurations Info" contains one entry. A red arrow points to the domain name column, and a white callout box with the text "Copy Domain name" is positioned next to it.

Name	Domain	Status	Date updated
iot:Data-ATS	afdh67wdpeoh9-ats.iot.ap-southeast-1.amazonaws.com	Enabled	August 03, 2025, 22:34:15 (U

Configure the Server field

Edit mqtt out node > Edit mqtt-broker node

Delete Cancel Update

⚙ Properties

🔑 Name AWS New Endpoint

Connection Security Messages

🌐 Server ah9sz9lh34wrz-ats.iot.ap-southeast-1.amazon Port 8883

☒ Connect automatically

☒ Use TLS none

⚙ Protocol MQTT V5

🔑 Client ID Leave blank for auto generated

💓 Keep Alive 60

📄 Session ☒ Use clean start

Session Expiry (secs)

User Properties none

- 1) Enter the name
- 2) Paste the copied domain name and set the port to 8883
- 3) Tick Use TLS option
- 4) Change the Protocol to MQTT V5

Change Use TLS option

Edit mqtt out node > Edit mqtt-broker node

Delete Cancel Update

⚙ Properties

📁 Name AWS New Endpoint

Connection Security Messages

🌐 Server ah9sz9lh34wrz-ats.iot.ap-southeast-1.amazon Port 8883

☒ Connect automatically

☒ Use TLS TLS configuration  

⚙ Protocol MQTT V5

📁 Client ID Leave blank for auto generated

💓 Keep Alive 60

📁 Session ☒ Use clean start

Session Expiry (secs)

User Properties none

Click Edit



Edit tls-config config node

Input the 3 certificates

Edit mqtt out node > Add new mqtt-broker config node > **Edit tls-config node**

Delete Cancel Update

Properties

☐ Use key and certificates from local files

Certificate	Upload	3f9264afb3e15dbc95195d615cf2c33ac1a053d8b91e292bed7f62156dbb3bb9-certificate.p...	x
Private Key	Upload	3f9264afb3e15dbc95195d615cf2c33ac1a053d8b91e292bed7f62156dbb3bb9-private.pem....	x
Passphrase	private key passphrase (optional)		
CA Certificate	Upload	AmazonRootCA1.pem	x

☒ Verify server certificate

Server Name

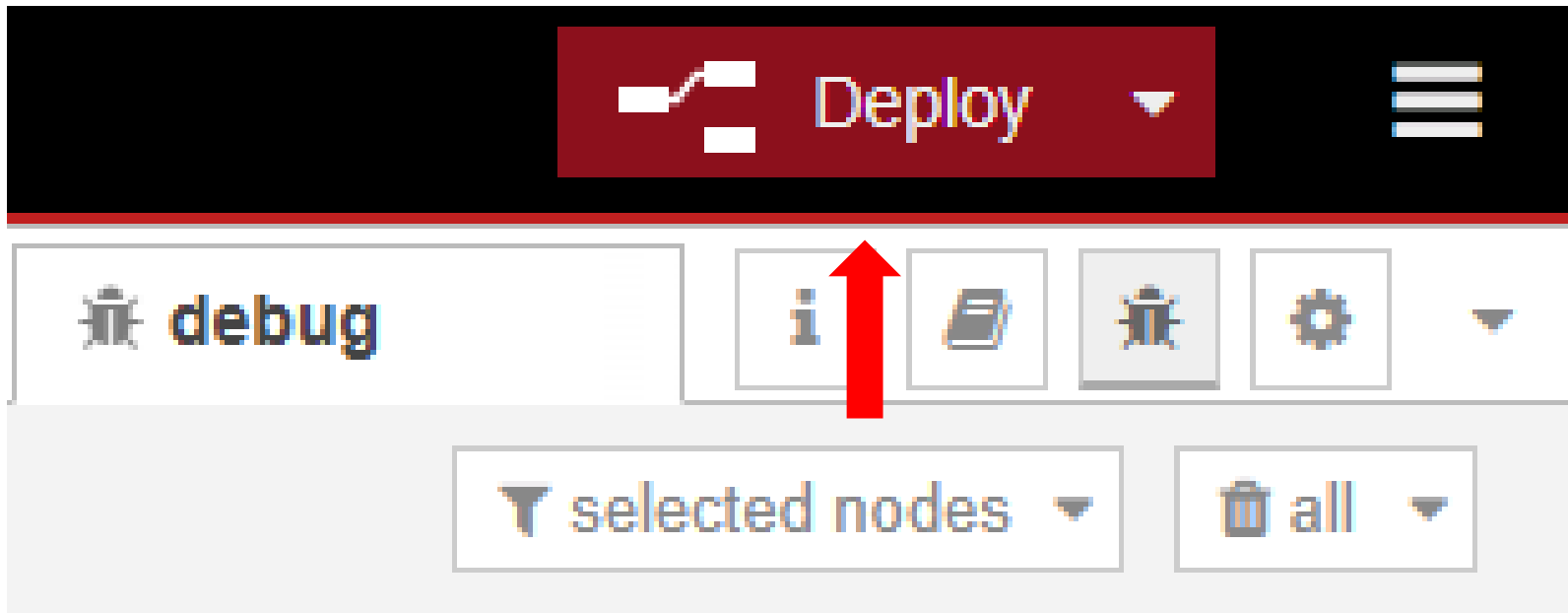
ALPN Protocol

Name

3 previously downloaded Certificates:

- 1) Certificate
- 2) Private
- 3) AmazonRootCA1

Click Deploy



Lambda

Lambda Home Page

The screenshot shows the AWS Lambda console interface. On the left is a dark sidebar with navigation links: 'Lambda' (selected), 'Dashboard', 'Applications', 'Functions', 'Additional resources' (expanded, showing 'Code signing configurations', 'Event source mappings', 'Layers', 'Replicas'), and 'Related AWS resources' (expanded, showing 'Step Functions state machines'). The main content area has a dark background. At the top, it says 'Compute' and 'AWS Lambda lets you run code without thinking about servers.' Below this is a paragraph: 'You pay only for the compute time that you consume — there is no charge when your code is not running. With Lambda, you can run code for virtually any type of application or backend service, all with zero administration.' To the right is a 'Get started' box with the text 'Author a Lambda function from scratch, or choose from one of many preconfigured examples.' and a yellow 'Create a function' button. At the bottom is a 'How it works' section with tabs for '.NET', 'Java', 'Node.js' (selected), 'Python', 'Ruby', and 'Custom runtime'. A yellow 'Run' button is next to the tabs. Below the tabs is a code editor showing a Node.js function handler:

```
1 * exports.handler = async (event) => {
2   console.log(event);
3   return 'Hello from Lambda!';
4 }
```

 To the right of the code editor is a 'Next: Lambda responds to events' button.

Compute

AWS Lambda

lets you run code without thinking about servers.

You pay only for the compute time that you consume — there is no charge when your code is not running. With Lambda, you can run code for virtually any type of application or backend service, all with zero administration.

Get started

Author a Lambda function from scratch, or choose from one of many preconfigured examples.

Create a function

How it works

Run Next: Lambda responds to events

.NET Java **Node.js** Python Ruby Custom runtime

```
1 * exports.handler = async (event) => {
2   console.log(event);
3   return 'Hello from Lambda!';
4 }
```

Click Create function

The screenshot shows the AWS Lambda console dashboard. On the left is a navigation menu with sections: **Lambda** (containing Dashboard, Applications, Functions), **Additional resources** (containing Code signing configurations, Event source mappings, Layers, Replicas), and **Related AWS resources** (containing Step Functions state machines). The main content area is titled **Resources for Asia Pacific (Singapore)** and displays four metrics: **Lambda function(s)** with a value of 0, **Code storage** with a value of 0 byte (0% of 75 GB), **Full account concurrency** with a value of 10, and **Unreserved account concurrency** with a value of 10. A red arrow points to the **Create function** button located to the right of the concurrency metrics. Below the resource metrics is a section titled **Top 10 functions** with a subtitle: "The charts below show the top 10 functions in each category from the last 3 hours in this AWS region." This section contains three charts: **Errors**, **Invocations**, and **Concurrent Executions**. Each chart shows a count on the y-axis (0 to 1) and a time range on the x-axis (17:40 to 20:40). All three charts display the message "No data available. Try adjusting the dashboard time range." On the right side of the dashboard, there is a sidebar with **Info** and **Tutorials** tabs. The **Tutorials** tab is active, showing a section titled **Create a simple web app** with a list of steps: "Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage" and "Invoke your function through its function URL". Below the list are links for [Learn more](#) and a **Start tutorial** button.

Lambda

- Dashboard
- Applications
- Functions

Additional resources

- Code signing configurations
- Event source mappings
- Layers
- Replicas

Related AWS resources

- Step Functions state machines

Resources for Asia Pacific (Singapore)

Create function

Lambda function(s)
0

Code storage
0 byte
(0% of 75 GB)

Full account concurrency
10

Unreserved account concurrency
10

Top 10 functions
The charts below show the top 10 functions in each category from the last 3 hours in this AWS region.

Errors

Count

1

No data available.
Try adjusting the dashboard time range.

0.5

0

17:40 20:40

Invocations

Count

1

No data available.
Try adjusting the dashboard time range.

0.5

0

17:40 20:40

Concurrent Executions

Count

1

No data available.
Try adjusting the dashboard time range.

0.5

0

17:40 20:40

Info **Tutorials**

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Start tutorial

Configure function

Create function Info
Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container**
Select a container image.

Basic information

Function name
Enter a name that describes the purpose of your function.
updateClassroom

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Node.js 22.x

Architecture Info
Choose the instruction set architecture you want for your function code.
☐ arm64
☒ x86_64

Permissions Info
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions
☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.
lambda_and_dynamo_access_role

View the lambda_and_dynamo_access_role role on the IAM console.

► **Additional configurations**
Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

Cancel Create function

Insert or select function information:

1. Name
2. Runtime
3. Architecture

Select Use an existing role and select the newly created role.

Click create function.

Insert the code

The screenshot displays the AWS Lambda console for a function named 'updateClassrom'. The top section, 'Function overview', includes tabs for 'Diagram' and 'Template', a diagram showing the function connected to an 'AWS IoT' trigger, and buttons for 'Add trigger', 'Add destination', 'Export to Infrastructure Composer', 'Download', 'Throttle', 'Copy ARN', and 'Actions'. The right sidebar provides details: 'Description' (empty), 'Last modified' (7 days ago), 'Function ARN' (arn:aws:lambda:ap-southeast-1:019662059667:function:updateClassrom), and 'Function URL' (empty). Below this is a tabbed interface with 'Code', 'Test', 'Monitor', 'Configuration', 'Aliases', and 'Versions'. The 'Code source' tab is active, showing a file explorer on the left with 'UPDATECLASSROOM' and 'index.mjs'. The main editor displays the code for 'index.mjs', which imports 'DynamoDBClient' and 'DynamoDBDocumentClient' from '@aws-sdk/client-dynamodb' and '@aws-sdk/lib-dynamodb' respectively. It initializes a client and document client, then defines an async handler that logs the event, validates its structure, and returns a 400 status code with a message if validation fails. A red arrow points to the code editor, and a text box below it says 'Refer to the Appendix for an example of code'.

updateClassrom

▼ Function overview Info

Diagram Template

updateClassrom

Layers (0)

AWS IoT (2)

+ Add trigger

+ Add destination

Export to Infrastructure Composer Download

Throttle Copy ARN Actions

Description

Last modified 7 days ago

Function ARN arn:aws:lambda:ap-southeast-1:019662059667:function:updateClassrom

Function URL Info

Code Test Monitor Configuration Aliases Versions

Code source Info

Open in Visual Studio Code Upload from

EXPLORER

UPDATECLASSROOM

index.mjs

DEPLOY

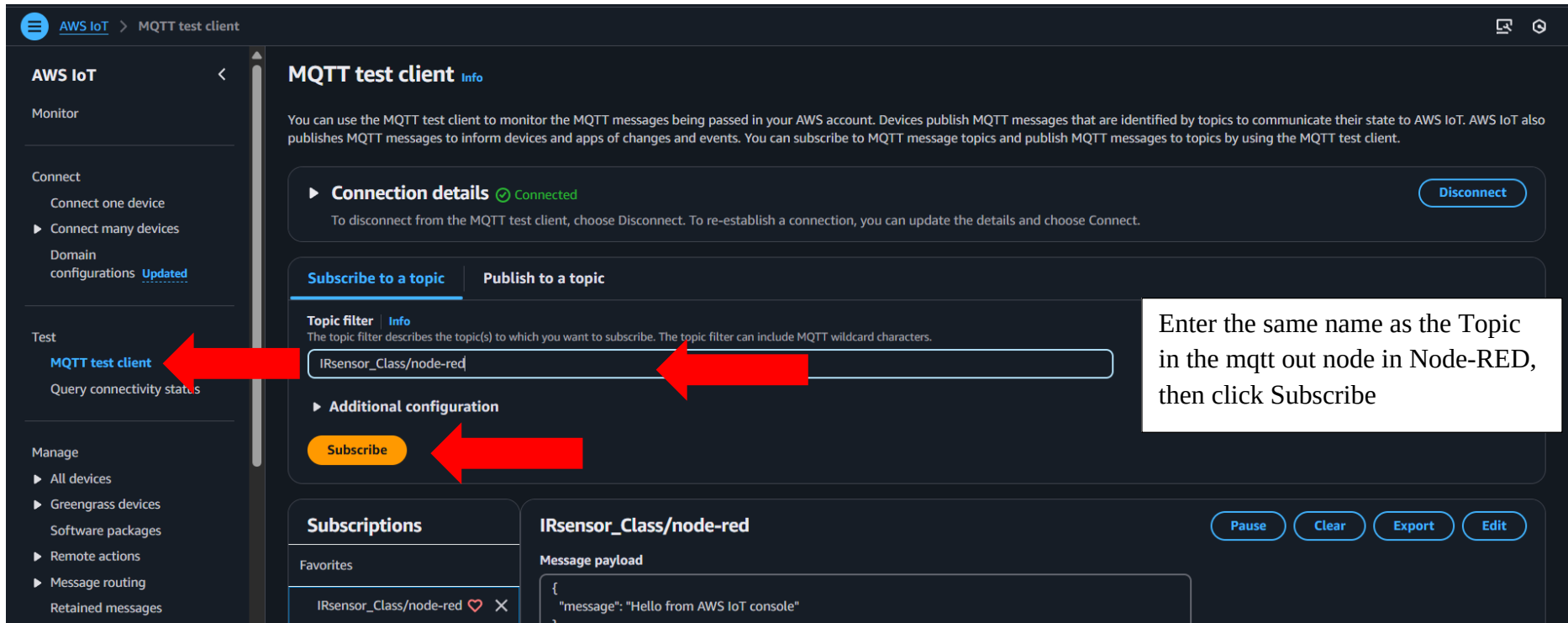
Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

```
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2 import { DynamoDBDocumentClient, UpdateCommand, PutCommand } from "@aws-sdk/lib-dynamodb";
3
4 const client = new DynamoDBClient({});
5 const db = DynamoDBDocumentClient.from(client);
6
7 export const handler = async (event) => {
8   console.log('Received event:', JSON.stringify(event, null, 2));
9   const { type, classId } = event;
10
11   if (!type || !classId) {
12     console.error('Validation Error: Missing "type" or "classId"');
13     return {
14       statusCode: 400,
15       body: JSON.stringify('Missing "type" or "classId" in the event.')
16     };
17   }
18 }
```

Refer to the Appendix for an example of code

Navigate to the MQTT test client under *Test* in AWS IoT Core



The screenshot shows the AWS IoT Core console with the MQTT test client interface. The left sidebar contains a navigation menu with sections: AWS IoT, Monitor, Connect, Domain configurations, Test, and Manage. The 'Test' section is expanded, and 'MQTT test client' is highlighted with a red arrow. The main panel shows the MQTT test client details, including a 'Connection details' section with a 'Disconnect' button. Below this, there are tabs for 'Subscribe to a topic' and 'Publish to a topic'. The 'Subscribe to a topic' tab is active, showing a 'Topic filter' input field with the text 'IRsensor_Class/node-red' and a red arrow pointing to it. Below the input field is an 'Additional configuration' section with a 'Subscribe' button and a red arrow pointing to it. A text box on the right side of the interface contains the instruction: 'Enter the same name as the Topic in the mqtt out node in Node-RED, then click Subscribe'. At the bottom, there is a 'Subscriptions' table with one entry: 'IRsensor_Class/node-red' with a heart icon and a close icon. To the right of the table, there is a 'Message payload' section with a text area containing a JSON object: { "message": "Hello from AWS IoT console" } and buttons for 'Pause', 'Clear', 'Export', and 'Edit'.

MQTT test client Info

You can use the MQTT test client to monitor the MQTT messages being passed in your AWS account. Devices publish MQTT messages that are identified by topics to communicate their state to AWS IoT. AWS IoT also publishes MQTT messages to inform devices and apps of changes and events. You can subscribe to MQTT message topics and publish MQTT messages to topics by using the MQTT test client.

► **Connection details** Connected Disconnect

To disconnect from the MQTT test client, choose Disconnect. To re-establish a connection, you can update the details and choose Connect.

Subscribe to a topic **Publish to a topic**

Topic filter Info
The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.

IRsensor_Class/node-red

► **Additional configuration**

Subscribe

Subscriptions

IRsensor_Class/node-red Pause Clear Export Edit

Message payload

```
{
  "message": "Hello from AWS IoT console"
}
```

Enter the same name as the Topic in the mqtt out node in Node-RED, then click Subscribe

Navigate to the Rules under the *Message routing*

The screenshot shows the AWS IoT console interface. The breadcrumb navigation at the top indicates the path: **AWS IoT** > **Message routing** > **Rules**. A green notification banner at the top states "Successfully created policy IRsensor_policy." with a "View policy" link.

The left sidebar contains several sections: "Connect", "Test", and "Manage". Under the "Manage" section, "Message routing" is expanded, and the "Rules" link is highlighted with a red arrow.

The main content area displays the "Rules (0)" page. It includes a "Find rules" search bar and a table with columns: Name, Status, Rule topic, and Created date. The table is currently empty, showing a "No rules" message: "You don't have any rules in ap-southeast-1." with a "Create rule" button. In the top right of this section, there are buttons for "Activate", "Deactivate", "Edit", "Delete", and a yellow "Create rule" button, which is pointed to by a red arrow.

A text box in the bottom right corner of the screenshot says "Click Create rule".

Input Rule name for the first rule

Step 1

Specify rule properties

Step 2

Configure SQL statement

Step 3

Attach rule actions

Step 4

Review and create

Specify rule properties Info

A rule resource contains a list of actions based on the MQTT topic stream.

Rule properties

Rule name

store_Classroom_Data

Enter an alphanumeric string that can also contain underscore () characters, but no spaces.

Rule description - optional

Enter a description to provide additional details about the rule to others.

A description of your new rule

▼ Tags - optional

No tags associated with the resource.

Add new tag

You can add up to 50 tags.

Cancel

Next

Input the SQL queries

Configure SQL statement

Step 3

Attach rule actions

Step 4

Review and create

SQL statement Info

SQL version

The version of the SQL rules engine to use when evaluating the rule.

2016-03-23

SQL statement

Enter a SQL statement using the following: SELECT <Attribute> FROM <Topic Filter> WHERE <Condition>. For example: SELECT temperature FROM 'iot/topic' WHERE temperature > 50. To learn more, see [AWS IoT SQL Reference](#).

1 SELECT type AS type,

2 classID AS classId,

3 { "currentCount": occupancy } AS data

4 FROM 'IRsensor_Class/node-red'

5 WHERE type = 'classroom_update'

Refer to the Appendix for the code
example of code

SQL Ln 5, Col 1

Errors: 0 Warnings: 0

Cancel

Previous

Next

Configure the Rule actions as below

Step 2

Configure SQL statement

Step 3

Attach rule actions

Step 4

Review and create

An action routes data to a specific AWS service.

SQL statement

Back

SELECT type AS type, classID AS classId, { "currentCount": occupancy } AS data FROM 'IRsensor_Class/node-red' WHERE type = 'classroom_update'

Rule actions Info

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that occur when messages arrive, like storing them in a database, invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

Lambda

Send a message to a Lambda function

Remove

Lambda function Info

updateClassrom

↺ ↻

Create a Lambda function ?

Lambda function version

\$LATEST

↻

Add rule action

Error action - *optional*

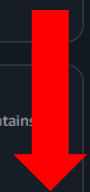
You can optionally set an action that will be executed when something goes wrong with processing your rule. If two rule actions in the same rule fail, the error action receives one message that contains both errors.

Add error action

Cancel

Previous

Next



Click Create

Step 3

● Attach rule actions

Step 4

● **Review and create**

Rule properties

Name

store_Classroom_Data

Description

-

Step 2: SQL statement

Edit

SQL statement

SQL version

2016-03-23

SQL query

SELECT type AS type, classID AS classId, { "currentCount": occupancy } AS data FROM 'IRsensor_Class/node-red' WHERE type = 'classroom_update'

Step 3: Rule actions

Edit

Actions

Lambda function

arn:aws:lambda:ap-southeast-1:019662059667:function:updateClassrom [🔗](#)

Lambda function version

\$LATEST

Error action

No error action

Cancel

Previous

Create

Input Rule name for the second rule

Step 1

Specify rule properties

Step 2

Configure SQL statement

Step 3

Attach rule actions

Step 4

Review and create

Specify rule properties [Info](#)

A rule resource contains a list of actions based on the MQTT topic stream.

Rule properties

Rule name

Enter an alphanumeric string that can also contain underscore () characters, but no spaces.

Rule description - optional
Enter a description to provide additional details about the rule to others.

Tags - optional
No tags associated with the resource.

[Add new tag](#)
You can add up to 50 tags.

[Cancel](#) [Next](#)

Input the SQL queries

Step 2
● **Configure SQL statement**
○ Step 3
○ Attach rule actions
○ Step 4
○ Review and create

Add a simplified SQL syntax to filter messages received on an MQTT topic and push the data elsewhere.

SQL statement info

SQL version
The version of the SQL rules engine to use when evaluating the rule.
2016-03-23 ▼

SQL statement
Enter a SQL statement using the following: SELECT <Attribute> FROM <Topic Filter> WHERE <Condition>. For example: SELECT temperature FROM 'iot/topic' WHERE temperature > 50. To learn more, see [AWS IoT SQL Reference](#).

```
1 SELECT type AS type,  
2     classID AS classId,  
3     { "action": action, "timestamp": timestamp } AS data  
4 FROM 'IRsensor_Class/node-red'  
5 WHERE type = 'occupancy_event'
```

SQL Ln 5, Col 31 | Errors: 0 | Warnings: 0

Cancel Previous **Next**

Refer to the Appendix for the code

Configure the Rule actions as below

Step 2

Configure SQL statement

Step 3

Attach rule actions

Step 4

Review and create

An action routes data to a specific AWS service.

SQL statement

Back

SELECT type AS type, classID AS classId, { "currentCount": occupancy } AS data FROM 'IRsensor_Class/node-red' WHERE type = 'classroom_update'

Rule actions Info

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that occur when messages arrive, like storing them in a database, invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

Lambda

Send a message to a Lambda function

Remove

Lambda function Info

updateClassrom

↺ ↻

Create a Lambda function ?

Lambda function version

\$LATEST

↻

Add rule action

Error action - *optional*

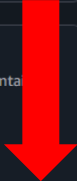
You can optionally set an action that will be executed when something goes wrong with processing your rule. If two rule actions in the same rule fail, the error action receives one message that contains both errors.

Add error action

Cancel

Previous

Next



Click Create

Step 4

Review and create

store_OccupancyEvent_Data

Description

-

Step 2: SQL statement

Edit

SQL statement

SQL version

2016-03-23

SQL query

SELECT type AS type, classID AS classId, { "action": action, "timestamp": timestamp } AS data FROM 'IRsensor_Class/node-red' WHERE type = 'occupancy_event'

Step 3: Rule actions

Edit

Actions

Lambda function

arn:aws:lambda:ap-southeast-1:019662059667:function:updateClassrom

Lambda function version

\$LATEST

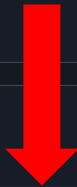
Error action

No error action

Cancel

Previous

Create



Simulate Node-RED to test its connection to IoT Core

Subscriptions

Favorites

IRsensor_Class/node-red  

All subscriptions

IRsensor_Class/node-red

Message payload

```
{
  "message": "Hello from AWS IoT console"
}
```

► Additional configuration

Publish

Successful connection confirmed by message

▼ IRsensor_Class/node-red

September 12, 2025, 15:35:36 (UTC+0800)

```
{
  "Name": "IR sensor for entry detection (N001)",
  "type": "occupancy_event",
  "action": "Enter",
  "classID": "N001",
  "timestamp": "12/09/2025, 15:35:36.558"
}
```

► Properties

▼ IRsensor_Class/node-red

September 12, 2025, 15:35:36 (UTC+0800)

```
{
  "Name": "IR sensor for entry detection (N001)",
  "type": "classroom_update",
  "classID": "N001",
  "occupancy": 14,
  "timestamp": "12/09/2025, 15:35:36.558"
}
```

DynamoDB

Create Table

Database

Amazon DynamoDB

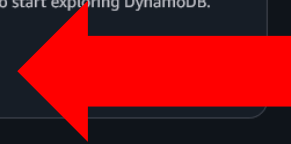
A fast and flexible NoSQL database service for any scale

DynamoDB is a fully managed, key-value, and document database that delivers single-digit-millisecond performance at any scale.

Get started

Create a new table to start exploring DynamoDB.

Create table



Pricing

Insert Table Information

aws

Search [Alt+S]

Asia Pacific (Singapore)

Console-to-Code

DynamoDB

Tables

Create table

1

Create table

Table details info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

Classroom

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

classIdString

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key nameString

1 to 255 characters and case sensitive.

Default settings

The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose 'Customize settings'.

Customize settings

Use these advanced features to make DynamoDB work better for your needs.

Default table settings

These are the default settings for your new table. You can change some of these settings after creating the table.

Setting	Value	Editable after creation
Table class	DynamoDB Standard	Yes
Capacity mode	On-demand	Yes
Maximum read capacity units	-	Yes

Insert table information:

1. Table name

2. Table primary key (*dubbed partition key*)

Repeat this twice for ClassSchedule(classId) and OccupanyEvents(classId)

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Setup Database for Querying

Create Role for Lambda

The screenshot shows the AWS IAM Dashboard. A red arrow points from the 'Roles' link in the left sidebar to the 'Roles' column in the 'IAM resources' table. Another red arrow points from the bottom of the 'IAM resources' table towards the 'What's new' section.

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

- User groups
- Users
- Roles**
- Policies
- Identity providers
- Account settings
- Root access management New

Access reports

- Access Analyzer
 - Resource analysis New
 - Unused access
 - Analyzer settings
- Credential report
- Organization activity
- Service control policies
- Resource control policies New

IAM Identity Center

AWS Organizations

IAM Dashboard info

Security recommendations 1

- Root user has MFA
Having multi-factor authentication (MFA) for the root user improves security for this account.
- Add MFA for yourself
Add multi-factor authentication (MFA) for yourself to improve security for this account. Add MFA
- Your user, jeremy, does not have any active access keys that have been unused for more than a year.
Deactivating or deleting unused access keys improves security.

IAM resources

Resources in this AWS Account

User groups	Users	Roles	Policies	Identity providers
1	3	14	9	0

What's new View all

- Amazon Bedrock introduces API keys for streamlining deployment. 1 month ago
- AWS Service Reference Information now supports actions for service actions. 2 months ago
- AWS expands resource control policies (RCPs) support to additional services. 2 months ago
- AWS IAM now enforces MFA for root users across all account types. 3 months ago

AWS Account

Account ID
019662059667

Account Alias
Create

Sign-in URL for IAM users in this account
<https://019662059667.signin.aws.amazon.com/console>

Quick Links

[My security credentials](#)

Manage your access keys, multi-factor authentication (MFA) and other credentials.

Tools

[Policy simulator](#)

The simulator evaluates the policies that you choose and determines the effective permissions for each of the actions that you specify.

Additional information

[Security best practices in IAM](#)

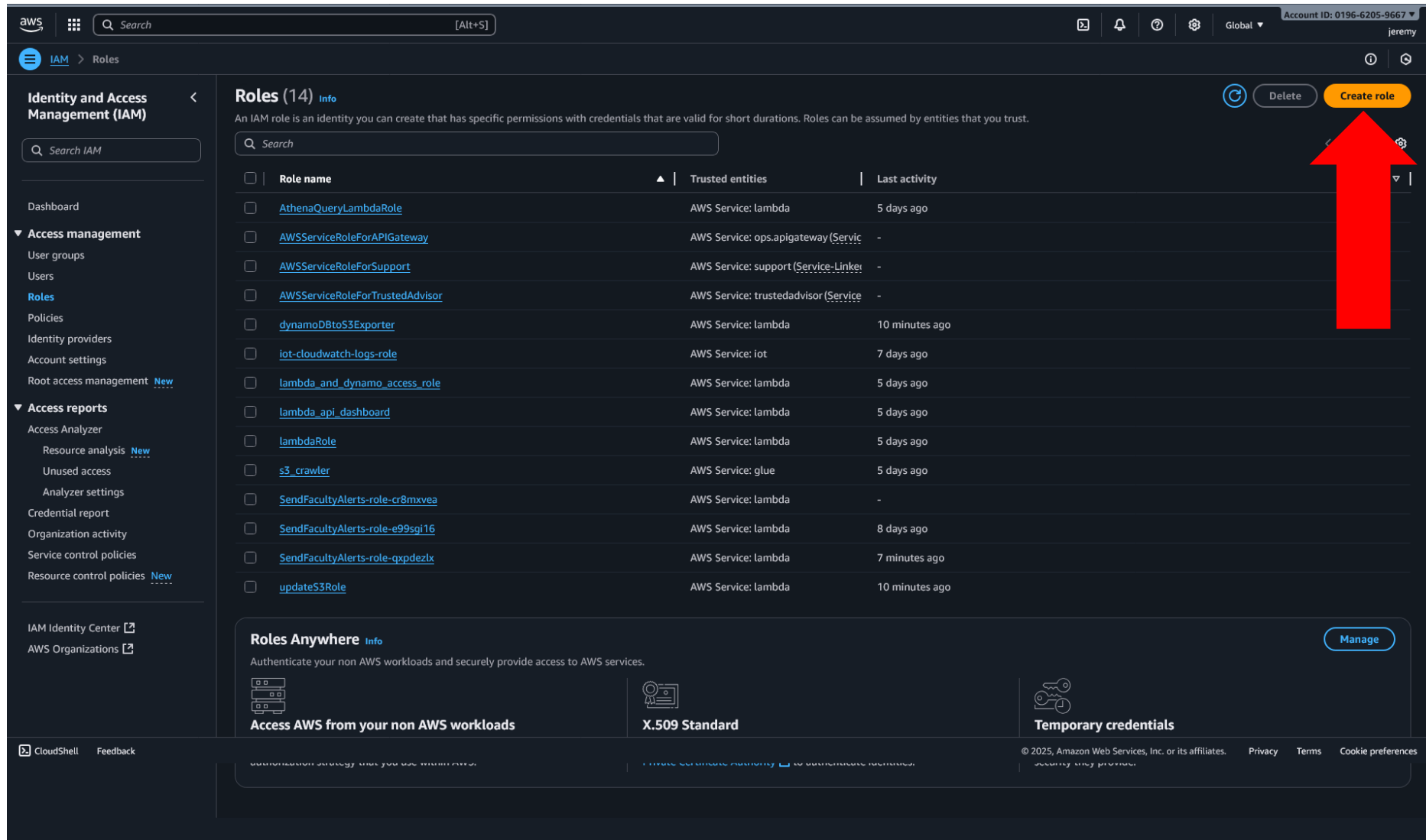
[IAM documentation](#)

[Videos, blog posts, and additional resources](#)

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Navigate to the role section and press create role



The screenshot shows the AWS IAM console interface. On the left is a navigation sidebar with sections for 'Identity and Access Management (IAM)' and 'Access management'. The main area is titled 'Roles (14)' and contains a table of existing roles. A large red arrow points to the 'Create role' button in the top right corner of the main area.

Roles (14)

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Role name	Trusted entities	Last activity
AthenaQueryLambdaRole	AWS Service: lambda	5 days ago
AWSServiceRoleForAPIGateway	AWS Service: ops.apigateway (Service-Linker)	-
AWSServiceRoleForSupport	AWS Service: support (Service-Linker)	-
AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linker)	-
dynamoDBtoS3Exporter	AWS Service: lambda	10 minutes ago
iot-cloudwatch-logs-role	AWS Service: iot	7 days ago
lambda_and_dynamo_access_role	AWS Service: lambda	5 days ago
lambda_api_dashboard	AWS Service: lambda	5 days ago
lambdaRole	AWS Service: lambda	5 days ago
s3_crawler	AWS Service: glue	5 days ago
SendFacultyAlerts-role-cr8mxvea	AWS Service: lambda	-
SendFacultyAlerts-role-e99sgj16	AWS Service: lambda	8 days ago
SendFacultyAlerts-role-qxpdezlx	AWS Service: lambda	7 minutes ago
updateS3Role	AWS Service: lambda	10 minutes ago

Roles Anywhere

Authenticate your non AWS workloads and securely provide access to AWS services.

Access AWS from your non AWS workloads

X.509 Standard

Temporary credentials

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Select Lambda under use case and click next

The screenshot shows the AWS IAM console interface for creating a new role. The breadcrumb navigation at the top indicates the path: IAM > Roles > Create role. The account ID 0196-6205-9667 and the user 'jeremy' are visible in the top right corner.

Step 1: Select trusted entity

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

Lambda

Choose a use case for the specified service.

Use case

- ☒ **Lambda**
Allows Lambda functions to call AWS services on your behalf.

At the bottom right of the main content area, there are two buttons: 'Cancel' and 'Next'. A large red arrow points directly to the 'Next' button.

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws

Search

[Alt+S]

Global

Account ID: 0196-6205-9667

jeremy

IAM > Roles > Create role

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Add permissions

Permissions policies (3/1082)

Choose one or more policies to attach to your new role.

Search

Filter by Type

All types

Policy name	Type
☐ AdministratorAccess	AWS managed - job function
☐ AdministratorAccess-Amplify	AWS managed
☐ AdministratorAccess-AWSElasticBeanstalk	AWS managed
☐ AIOpsAssistantPolicy	AWS managed
☐ AIOpsConsoleAdminPolicy	AWS managed
☐ AIOpsOperatorAccess	AWS managed
☐ AIOpsReadOnlyAccess	AWS managed
☐ AlexaForBusinessDeviceSetup	AWS managed
☐ AlexaForBusinessFullAccess	AWS managed
☐ AlexaForBusinessGatewayExecution	AWS managed
☐ AlexaForBusinessLifeseizeDelegatedAccessPolicy	AWS managed
☐ AlexaForBusinessPolyDelegatedAccessPolicy	AWS managed
☐ AlexaForBusinessReadOnlyAccess	AWS managed
☐ AmazonAPIGatewayAdministrator	AWS managed
☐ AmazonAPIGatewayInvokeFullAccess	AWS managed
☐ AmazonAPIGatewayPushToCloudWatchLogs	AWS managed

Select these permissions:

1. AmazonDynamoDBFullAccess_v2

2. AmazonS3FullAccess

3. AWSLambdaBasicExecutionRole

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Give the role a name and a description

aws

Search

[Alt+S]

Global

Account ID: 0196-6205-9667

jeremy

IAM > Roles > Create role

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

Maximum 64 characters. Use alphanumeric and '+,=,@,-' characters.

Description
Add a short explanation for this role.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/\[\]!#\$%^&*~"

Step 1: Select trusted entities

Edit

Trust policy

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "sts:AssumeRole"  
8       ],  
9       "Principal": {  
10        "Service": [  
11          "lambda.amazonaws.com"  
12        ]  
13      }  
14    }  
15  ]  
16 }
```

Edit

Step 2: Add permissions

Edit

Permissions policy summary

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Create Lambda Function

The screenshot displays the AWS Lambda console interface for the Asia Pacific (Singapore) region. At the top, the navigation bar includes the AWS logo, a search bar, and user information. The main content area is divided into several sections:

- Resources for Asia Pacific (Singapore):** This section shows four key metrics: Lambda function(s) at 7, Code storage at 4.5 MB (0% of 75 GB), Full account concurrency at 10, and Unreserved account concurrency at 10. A prominent red arrow points to the 'Create function' button in the top right corner of this section.
- Top 10 functions:** This section provides a summary of the top 10 functions in each category from the last 3 hours.
- Account-level metrics:** This section displays various performance metrics across all Lambda functions in the region. It includes a time range selector (1h, 3h, 12h, 1d, 3d, 1w, Custom) and a UTC timezone dropdown. The metrics shown are:
 - Error count and success rate:** A chart showing errors (max: -) and success rate (min: -%) over time.
 - Throttles:** A chart showing the count of throttles (max: -) over time.
 - Invocations:** A chart showing the count of invocations (sum: -) over time.
 - Duration:** A chart showing the duration in milliseconds over time.
 - Total concurrent executions:** A chart showing the count of total concurrent executions over time.
 - Unreserved concurrent executions:** A chart showing the count of unreserved concurrent executions over time.

The bottom of the console features a footer with links to CloudShell, Feedback, and a copyright notice for Amazon Web Services, Inc. or its affiliates, along with links to Privacy, Terms, and Cookie preferences.

Create function [Info](#)

Choose one of the following options to create your function.

☒ Author from scratch

Start with a simple Hello World example.

☐ Use a blueprint

Build a Lambda application from sample code and configuration presets for common use cases.

☐ Container image

Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

dynamoDBtoS3Exporter

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Node.js 22.x

Architecture [Info](#)

Choose the instruction set architecture you want for your function code.

☐ arm64

☒ x86_64

Permissions [Info](#)

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ Change default execution role

Execution role

Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions

☒ Use an existing role

☐ Create a new role from AWS policy templates

Existing role

Choose an existing role that you've created. The role must have permission to upload logs to Amazon CloudWatch Logs.

dynamoDBtoS3Exporter

Insert or select function information:

1. Name

2. Runtime

3. Architecture

Select Use an existing role and select the newly created role.

Click create function.

Navigate to the newly created function and insert the code.

The screenshot displays the AWS Lambda console interface for a function named **dynamoDBtoS3Exporter**. The top navigation bar shows the AWS logo, a search bar, and the account ID **0196-6205-9667** for the user **jeremy**. The breadcrumb trail indicates the path: **Lambda > Functions > dynamoDBtoS3Exporter**.

The function overview section includes tabs for **Diagram** and **Template**. The **Diagram** tab shows a visual representation of the function's configuration: a **dynamoDBtoS3Exporter** function icon connected to a **DynamoDB** trigger icon. The function has **1** layer and the trigger has **3** destinations. Buttons for **Throttle**, **Copy ARN**, **Actions**, **Export to Infrastructure Composer**, and **Download** are visible. A **+ Add destination** button is also present.

The **Description** section on the right provides details about the function: **Last modified** (2 weeks ago), **Function ARN** (`arn:aws:lambda:ap-southeast-1:019662059667:function:dynamoDBtoS3Exporter`), and **Function URL** (with an **Info** link).

The **Code source** section is active, showing a text box with the instruction **Refer to the Appendix for the code**. Below this, the **Code source** tab is selected, displaying the **index.mjs** file. The code is as follows:

```
1 import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";
2 import { unmarshall } from "@aws-sdk/util-dynamodb";
3
4 const s3 = new S3Client({});
5 const BUCKET_NAME = 'utility-classroom-data-lake'; // Your bucket name
6
7 export const handler = async (event) => {
8   for (const record of event.Records) {
9     if (record.eventName === 'INSERT' || record.eventName === 'MODIFY') {
10       const newImage = record.dynamodb.NewImage;
11
12       // Use the unmarshall function directly on the NewImage object
```

The bottom of the console shows the **CloudShell** icon, a **Feedback** link, and the copyright notice: **© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences**.

Click on the add trigger button and select trigger to be DynamoDB.

Add trigger

Trigger configuration [Info](#)

DynamoDB
aws database event-source-mapping nosql polling

DynamoDB table
Choose or enter the ARN of a DynamoDB table.

arn:aws:dynamodb:ap-southeast-1:019662059667:table/Classroom

Event poller configuration

☒ **Activate trigger**
Select to activate the trigger now. Keep unchecked to create the trigger in a deactivated state for testing (recommended).

☐ **Enable metrics**
Monitor your event source with metrics. You can view those metrics in CloudWatch console. Enabling this feature incurs additional costs. [Learn more](#)

Batch size [Info](#)
The maximum number of records in each batch to send to the function.

100

Starting position [Info](#)
The position in the stream to start reading from.

Latest

Batch window - optional
The maximum amount of time to gather records before invoking the function, in seconds.

► **Additional settings**

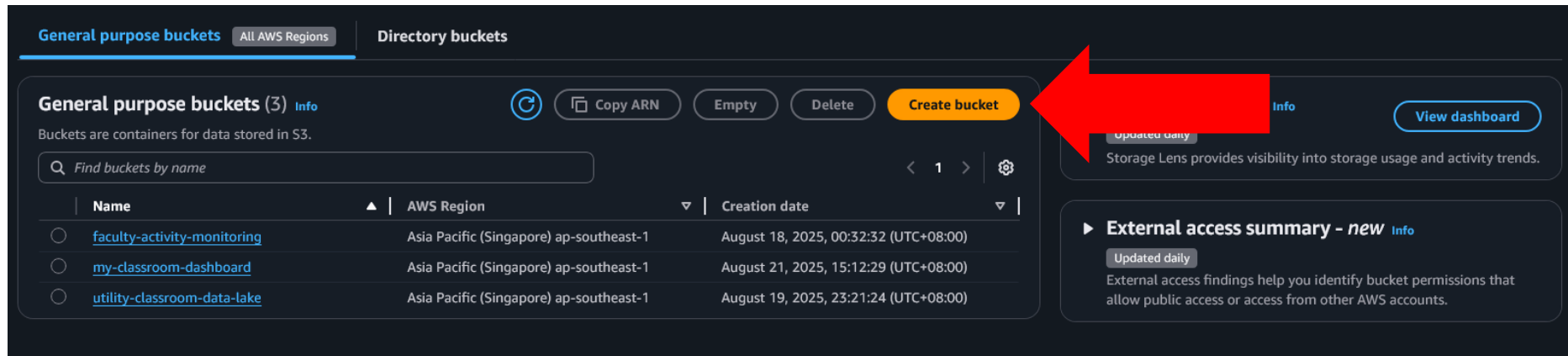
In order to read from the DynamoDB trigger, your execution role must have proper permissions.

[Cancel](#) [Add](#)

Repeat this for the other two tables.

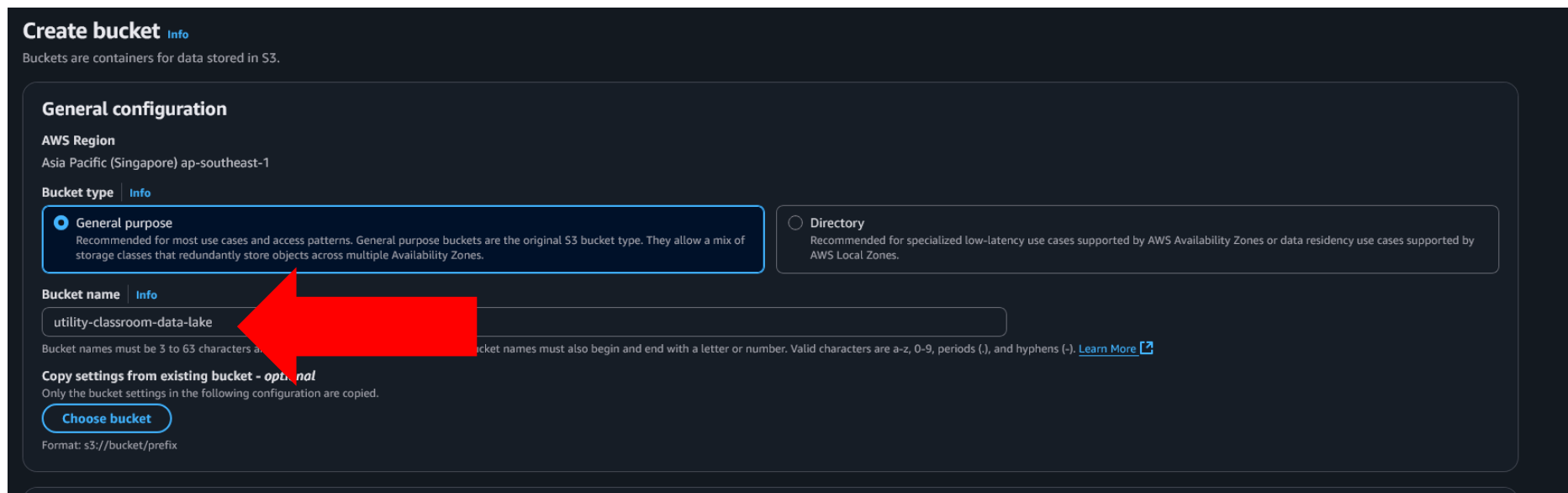
S3

Navigate to s3 bucket and press create bucket.



The screenshot shows the AWS S3 console interface. At the top, there are tabs for 'General purpose buckets' and 'Directory buckets'. Below the tabs, there's a section for 'General purpose buckets (3)' with an 'Info' link. It includes a search bar 'Find buckets by name' and a table of existing buckets. A red arrow points to the 'Create bucket' button in the top right of the bucket list section.

Name	AWS Region	Creation date
faculty-activity-monitoring	Asia Pacific (Singapore) ap-southeast-1	August 18, 2025, 00:32:32 (UTC+08:00)
my-classroom-dashboard	Asia Pacific (Singapore) ap-southeast-1	August 21, 2025, 15:12:29 (UTC+08:00)
utility-classroom-data-lake	Asia Pacific (Singapore) ap-southeast-1	August 19, 2025, 23:21:24 (UTC+08:00)



The screenshot shows the 'Create bucket' page in the AWS S3 console. It includes a 'General configuration' section with 'AWS Region' set to 'Asia Pacific (Singapore) ap-southeast-1'. Under 'Bucket type', the 'General purpose' option is selected. Below this, the 'Bucket name' is set to 'utility-classroom-data-lake'. A red arrow points to the 'General purpose' bucket type and the bucket name field. At the bottom, there's a 'Choose bucket' button and a format note: 'Format: s3://bucket/prefix'.

General configuration

AWS Region
Asia Pacific (Singapore) ap-southeast-1

Bucket type [Info](#)

- ☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.
- ☐ **Directory**
Recommended for specialized low-latency use cases supported by AWS Availability Zones or data residency use cases supported by AWS Local Zones.

Bucket name [Info](#)
utility-classroom-data-lake

Bucket names must be 3 to 63 characters and must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn More](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

AWS Glue

Create an IAM role

s3_crawler Info

Allows Glue to call AWS services on your behalf.

Delete

Edit

Summary

Creation date
August 19, 2025, 23:32 (UTC+08:00)

Last activity
✔ 5 days ago

ARN
arn:aws:iam::019662059667:role/s3_crawler

Maximum session duration
1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies Info

You can attach up to 10 managed policies.

↺

Simulate

Remove

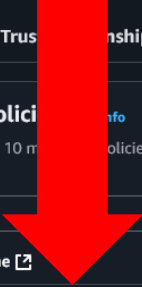
Add permissions

Search

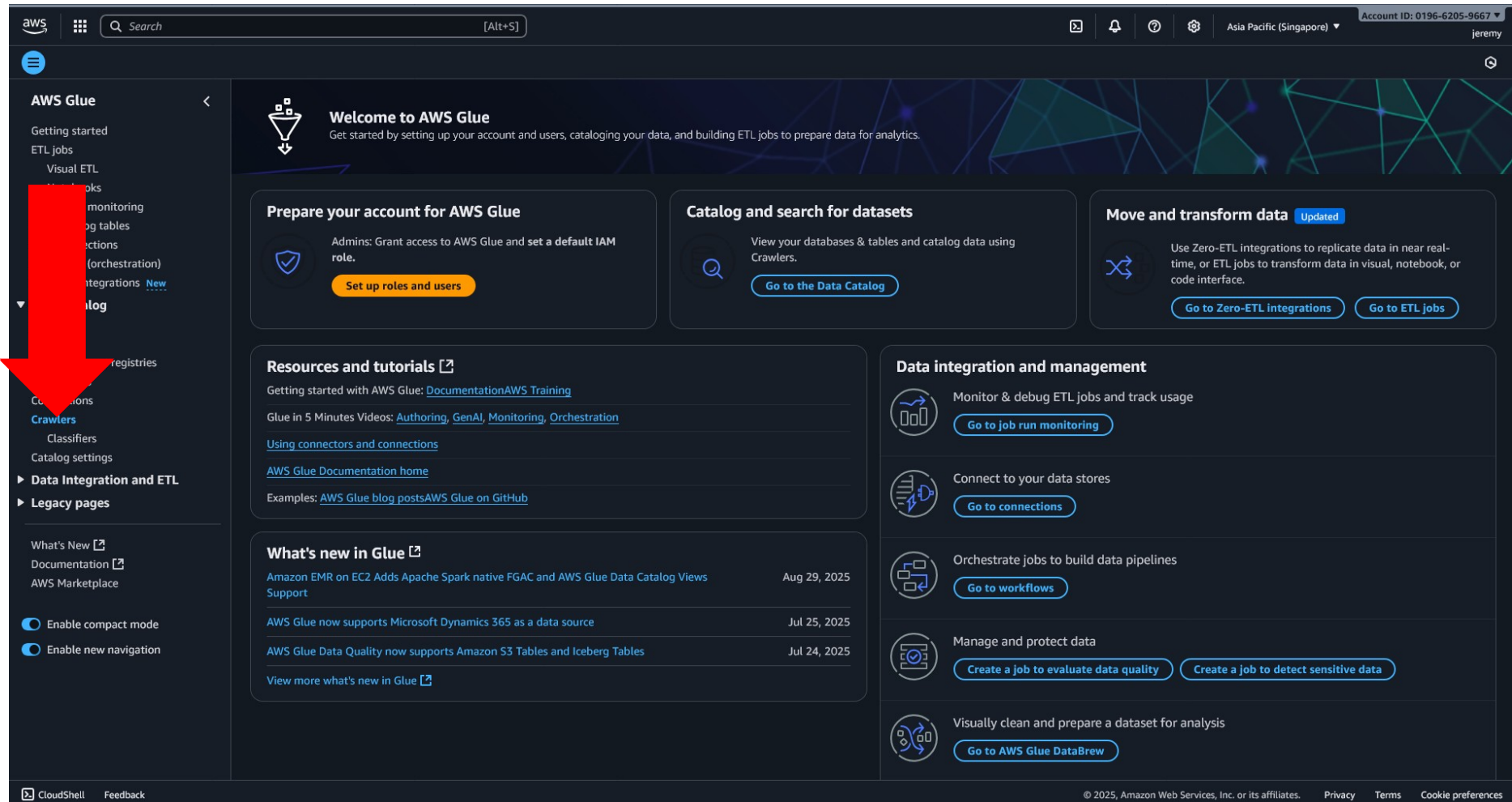
Filter by Type
All types

< 1 > ⚙

☐	Policy name	Type	Attached entities
☐	 AmazonS3FullAccess	AWS managed	6
☐	 AWSGlueServiceRole	AWS managed	1
☐	 CloudWatchLogsFullAccess	AWS managed	1



Navigate to the crawler section of the AWS Glue



The screenshot shows the AWS Glue console interface. A large red arrow points to the 'Crawlers' link in the left-hand navigation menu. The main content area displays a 'Welcome to AWS Glue' message and several action cards for account setup, cataloging, and data transformation. The right-hand sidebar contains sections for 'Data integration and management' and 'What's new in Glue'.

AWS Glue

- Getting started
- ETL jobs
- Visual ETL
- Monitors
- Monitoring
- Log tables
- Connections
- Orchestration (orchestration)
- Integrations **New**
- Crawlers**
- Registries
- Classifiers
- Catalog settings
- **Data Integration and ETL**
- **Legacy pages**

Welcome to AWS Glue
Get started by setting up your account and users, cataloging your data, and building ETL jobs to prepare data for analytics.

Prepare your account for AWS Glue
Admins: Grant access to AWS Glue and set a default IAM role.
[Set up roles and users](#)

Catalog and search for datasets
View your databases & tables and catalog data using Crawlers.
[Go to the Data Catalog](#)

Move and transform data **Updated**
Use Zero-ETL integrations to replicate data in near real-time, or ETL jobs to transform data in visual, notebook, or code interface.
[Go to Zero-ETL integrations](#) [Go to ETL jobs](#)

Resources and tutorials
Getting started with AWS Glue: [Documentation](#)[AWS Training](#)
Glue in 5 Minutes Videos: [Authoring](#), [GenAI](#), [Monitoring](#), [Orchestration](#)
[Using connectors and connections](#)
[AWS Glue Documentation home](#)
Examples: [AWS Glue blog posts](#)[AWS Glue on GitHub](#)

What's new in Glue
[Amazon EMR on EC2 Adds Apache Spark native FGAC and AWS Glue Data Catalog Views Support](#) Aug 29, 2025
[AWS Glue now supports Microsoft Dynamics 365 as a data source](#) Jul 25, 2025
[AWS Glue Data Quality now supports Amazon S3 Tables and Iceberg Tables](#) Jul 24, 2025
[View more what's new in Glue](#)

Data integration and management
Monitor & debug ETL jobs and track usage
[Go to job run monitoring](#)
Connect to your data stores
[Go to connections](#)
Orchestrate jobs to build data pipelines
[Go to workflows](#)
Manage and protect data
[Create a job to evaluate data quality](#) [Create a job to detect sensitive data](#)
Visually clean and prepare a dataset for analysis
[Go to AWS Glue DataBrew](#)

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Create a crawler

Step 1

● Set crawler properties

Step 2

● Choose data sources and classifiers

Step 3

● Configure security settings

Step 4

● Set output and scheduling

Step 5

● Review and update

Set crawler properties

Crawler details [Info](#)

Name

Name can be up to 255 characters long. Some character set including control characters are prohibited.

Description - optional

Descriptions can be up to 2048 characters long.

► **Tags - optional**

Use tags to organize and identify your resources.

[Cancel](#) [Next](#)

Select the Classroom data lake as the S3 source.

Edit data source

Data source

Choose the source of data to be crawled.

S3

Network connection - optional

Optionally include a Network connection to connect with this S3 target. Note that each crawler is limited to one Network connection so any other S3 targets will also use the same connection (or none, if left blank).

Clear selection

Add new connection

S3 path

Browse for or enter an existing S3 path.

s3://utility-classroom-data-lake/Classroom

All folders and files contained in the S3 path are crawled. For example, type s3://MyBucket/MyFolder/ to crawl all objects in MyFolder within MyBucket.

Subsequent crawler runs

This field is a global field that affects all S3 data sources.

☒ Crawl all sub-folders

Crawl all folders again with every subsequent crawl.

☐ Crawl new sub-folders only

Only Amazon S3 folders that were added since the last crawl will be crawled. If the schemas are compatible, new partitions will be added to existing tables.

☐ Crawl based on events

Rely on Amazon S3 events to control what folders to crawl.

☐ Sample only a subset of files

☐ Exclude files matching pattern

Select the created IAM role

Configure security settings

IAM role [Info](#)

Existing IAM role

s3_crawler   [View](#) 

[Create new IAM role](#) [Update chosen IAM role](#)

Only IAM roles created by the AWS Glue console and have the prefix "AWSGlueServiceRole-" can be updated.

Lake Formation configuration - optional

Allow the crawler to use Lake Formation credentials for crawling the data source. [Learn more.](#) 

☐ Use Lake Formation credentials for crawling S3 data source

Checking this box will allow the crawler to use Lake Formation credentials for crawling the data source. If the data source is registered in another account, you must provide the registered account ID. Otherwise, the crawler will crawl only those data sources associated to the account. Only applicable to S3, Glue Catalog, Iceberg, and Hudi data sources.

► **Security configuration - optional**

Enable at-rest encryption with a security configuration.

[Cancel](#) [Previous](#) [Next](#)

Create a database and table name prefix to store the output.
Set the schedule according to preference.

Set output and scheduling

Output configuration [Info](#)

Target database

classroom_analytics 

[Clear selection](#) [Add database](#) 

Table name prefix - optional

classroom_

Maximum table threshold - optional

This field sets the maximum number of tables the crawler is allowed to generate. In the event that this number is surpassed, the crawl will fail with an error. If not set, the crawler will automatically generate the number of tables depending on the data schema.

Type a number greater than 0 

► **Advanced options**

Crawler schedule

You can define a time-based schedule for your crawlers and jobs in AWS Glue. The definition of these schedules uses the Unix-like [cron](#)  syntax. [Learn more](#) .

Frequency

On demand 

[Cancel](#) [Previous](#) [Next](#)

Repeat this for the other two tables.

Querying the Database

Create an IAM role

lambda_api_dashboard [Info](#) [Delete](#)

API to call backend for dashboard

Summary

Creation date
August 21, 2025, 14:57 (UTC+08:00)

Last activity
✓ 5 days ago

ARN
[arn:aws:iam::019662059667:role/lambda_api_dashboard](#)

Maximum session duration
1 hour

[Edit](#)

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies

You can attach up to 10 managed policies.

Filter by Type

All types

< 1 >

⚙

☐

Policy name [↗](#)

▲

Type

▼

Attached entities

▼

☐

[AmazonAthenaFullAccess](#)

AWS managed

3

☐

[AmazonS3FullAccess](#)

AWS managed

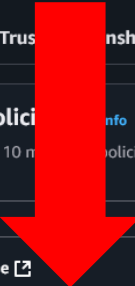
6

[Refresh](#)

[Simulate](#)

[Remove](#)

[Add permissions](#)



Create a lambda function and insert the necessary code.

dynamoDBtoS3Exporter [Throttle](#) [Copy ARN](#) [Actions](#) ▼

▼ **Function overview** [Info](#)

[Diagram](#) | [Template](#)

 **dynamoDBtoS3Exporter**

 Layers (1)

 **DynamoDB** (3)

[+ Add trigger](#)

[+ Add destination](#)

Description
-

Last modified
3 weeks ago

Function ARN
[Copy](#) arn:aws:lambda:ap-southeast-1:019662059667:function:dynamoDBtoS3Exporter

Function URL [Info](#)
-

[Export to Infrastructure Composer](#) [Download](#) ▼

Refer to the Appendix for the code

updateClassroom

ThrottleCopy ARNActions

▼ Function overviewInfo

DiagramTemplate

updateClassroom

Layers(0)

DynamoDB

AWS IoT(2)

+ Add trigger

SNS

+ Add destination

Export to Infrastructure Composer

Download

Description

-

Last modified

14 hours ago

Function ARN

arn:aws:lambda:ap-southeast-1:019662059667:function:updateClassroom

Function URL

Info

-

Code

Test

Monitor

Configuration

Aliases

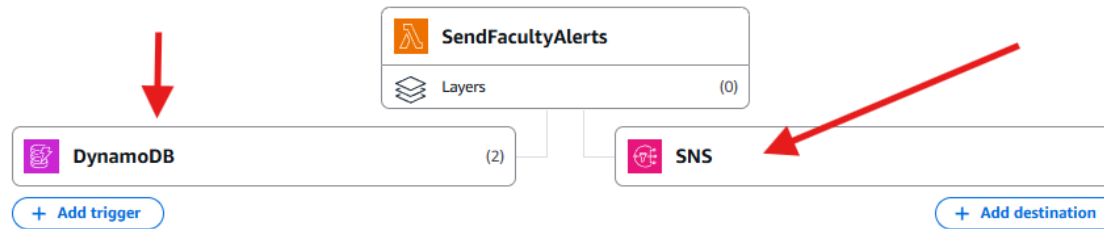
Versions

Refer to the Appendix for the code

SendFacultyAlerts

[Throttle](#)[Copy ARN](#)[Actions](#) ▼

▼ Function overview [Info](#)

[Export to Infrastructure Composer](#)[Download](#) ▼[Diagram](#)[Template](#)

Description

-

Last modified

20 hours ago

Function ARN

[arn:aws:lambda:ap-southeast-1:019662059667:function:SendFacultyAlerts](#)

Function URL [Info](#)

-

[Code](#)[Test](#)[Monitor](#)[Configuration](#)[Aliases](#)[Versions](#)

Refer to the Appendix for the code

Simple Notification Service

Navigate to the Amazon SNS Dashboard

The screenshot displays the Amazon SNS Dashboard. At the top, there's a navigation bar with the AWS logo, a search bar, and account information (Asia Pacific (Singapore), Account ID: 0196-6205-9667, user: calvin). The left sidebar shows the 'Amazon SNS' menu with options for 'Dashboard', 'Topics', 'Subscriptions', and 'Mobile'. The main content area features a blue banner for a 'New Feature' (High Throughput FIFO topics) and a 'Dashboard' section with three metrics: 'Resources for ap-southeast-1' (Topics: 1, Platform applications: 0, Subscriptions: 2). Below this is the 'Overview of Amazon SNS' section, which includes the 'Application-to-application (A2A)' subsection. This subsection describes Amazon SNS as a managed messaging service for decoupling publishers and subscribers. A diagram illustrates the A2A flow: a 'Publisher' (laptop and document icon) sends messages to 'Amazon SNS' (cloud icon). From there, messages go to an 'SNS topic' (stack of papers icon), which then goes to 'Message filtering and fanout' (funnel icon). A 'Dead-letter queue' (envelope icon) is shown as a side path from the SNS topic, with a note: 'If an endpoint is unavailable, messages can be held in a dead-letter queue for analysis or reprocessing'. Finally, messages are delivered to 'Subscribers' (envelope icon), which are listed as AWS Lambda, Amazon SQS, Amazon Kinesis Data Firehose, HTTP/HTTPS, and Email. The subscribers section notes: 'Receive messages in subscribing serverless functions, queues, microservices, delivery streams, and more'.

Amazon SNS

- Dashboard
- Topics
- Subscriptions
- ▼ Mobile
 - Push notifications
 - Text messaging (SMS)

Dashboard

Resources for ap-southeast-1

Topics	Platform applications	Subscriptions
1	0	2

▼ Overview of Amazon SNS

Application-to-application (A2A)

Amazon SNS is a managed messaging service that lets you decouple publishers from subscribers. This is useful for application-to-application messaging for microservices, distributed systems, and serverless applications. [Learn more](#)

Diagram: Application-to-application (A2A) messaging flow

```
graph LR; Publisher[Publisher: Publishers send messages from distributed systems, microservices, and other AWS services] --> SNS[Amazon SNS: Fully managed Pub/Sub messaging and event-driven computing services]; SNS --> Topic[SNS topic: Decouple message publishers from subscribers with topics]; Topic --> DLQ[Dead-letter queue: If an endpoint is unavailable, messages can be held in a dead-letter queue for analysis or reprocessing]; Topic --> Filter[Message filtering and fanout: Filter messages according to subscription filter policies and deliver them to subscribers]; Filter --> Subscribers[Subscribers: Receive messages in subscribing serverless functions, queues, microservices, delivery streams, and more];
```

Subscribers: AWS Lambda, Amazon SQS, Amazon Kinesis Data Firehose, HTTP/HTTPS, Email

Select the Topic

aws

Search

[Alt+S]

Asia Pacific (Singapore)

Account ID: 0196-6205-9667

calvin

Amazon SNS

Topics

Amazon SNS

Dashboard

Topics

Subscriptions

▼ Mobile

Push notifications

Text messaging (SMS)

New Feature

Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Topics (1)

Search

EditDeletePublish messageCreate topic

< 1 > ⚙

	Name	Type	ARN
☐	FacultyAlertsTopic	Standard	arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic

Create Topic

The screenshot shows the AWS Management Console interface for Amazon SNS. At the top, there's a navigation bar with the AWS logo, a search bar, and account information (Asia Pacific (Singapore), Account ID: 0196-4205-9667, user: calvin). Below the navigation bar, the left sidebar shows the 'Amazon SNS' menu with options like Dashboard, Topics, Subscriptions, and Mobile. The main content area is titled 'Topics (1)' and features a search bar and a table of topics. A blue banner at the top of the main area announces a new feature: 'Amazon SNS now supports High Throughput FIFO topics. Learn more'. The table lists one topic named 'FacultyAlertsTopic' with a type of 'Standard' and an ARN of 'arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic'. A red arrow points to the 'Create topic' button in the top right corner of the main content area.

Amazon SNS

Dashboard

Topics

Subscriptions

▼ Mobile

Push notifications

Text messaging (SMS)

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Topics (1)

Search

Edit Delete Publish message Create topic

Name	Type	ARN
FacultyAlertsTopic	Standard	arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic

Insert Topic name and details

  [AWS]

[Amazon SNS](#) > [Topics](#) > Create topic

New feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Create topic

Details

Type | [info](#)
Topic type cannot be modified after topic is created.

FIFO (first-in, first-out)

- Strictly-ordered message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

Standard

- Best-effort message ordering
- At-least-once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SNS, email, mobile application endpoints

Name [info](#)

Maximum 254 characters. Can include alphanumeric characters, hyphens (-) and underscores (_). FIFO topic names must end with "FIFO".

High throughput | [info](#)
Configure your FIFO topic for maximum throughput. Once enabled, this configuration cannot be modified.

☒ **Message group scope** (Recommended for highest throughput)

- Throughput: Maximum regional limits.
- Deduplication: Message deduplication is only verified within a message group.

☐ **Topic scope**

- Throughput: 1000 message per second and a bandwidth of 30MB per second.
- Deduplication: Message deduplication is verified on the entire FIFO topic.

Display name - optional | [info](#)
To use this topic with SNS subscriptions, enter a display name. Only the first 10 characters are displayed in an SNS message.

Maximum 100 characters.

Content-based message deduplication | [info](#)
Enables default message deduplication based on message content. If enabled, a deduplication ID must be provided for every publish request.
☐ Turn on message deduplication

► **Encryption - optional**

Amazon SNS provides in-transit encryption by default. Enabling server-side encryption adds at-rest encryption to your topic.

► **Access policy - optional** [info](#)

This policy defines who can access your topic. By default, only the topic owner can publish or subscribe to the topic.

► **Archive policy - new, optional** [info](#)

This policy tells Amazon SNS how long to store your messages so that they can be relevant to a subscription. By default, Amazon SNS does not retain your messages.

► **Delivery status logging - optional** [info](#)

These settings configure the logging of message delivery status to CloudWatch Logs.

► **Tags - optional**

A tag is a metadata label that you can assign to an Amazon SNS topic. Each tag consists of a key and an optional value. You can use tags to search and filter your topics and track your costs. [Learn more](#)

► **Active tracing - optional** [info](#)

Use AWS X-Ray active tracing for this topic to view its traces and service map in Amazon CloudWatch. Additional costs apply.

[Cancel](#) [Create topic](#)

Update the Access Policy

Details

Name
FacultyAlertsTopic

Type
Standard

Display name - optional [Info](#)
To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

Maximum 100 characters.

► **Encryption - optional**
Amazon SNS provides in-transit encryption by default. Enabling server-side encryption adds at-rest encryption to your topic.

▼ **Access policy - optional** [Info](#)
This policy defines who can access your topic. By default, only the topic owner can publish or subscribe to the topic.

JSON editor

```
14  "SNS:GetTopicAttributes",
15  "SNS:DeleteTopic",
16  "SNS:ListSubscriptionsByTopic",
17  "SNS:GetTopicAttributes",
18  "SNS:AddPermission",
19  "SNS:Subscribe"
20  ],
21  "Resource": "arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic",
22  "Condition": {
23    "StringEquals": {
24      "AWS:SourceAccount": "019662059667"
25    }
26  }
27  }
28  ]
```

←

▼ **Data protection policy - optional** [Info](#)
This policy defines which sensitive data to monitor and to prevent from being exchanged via your topic.

Configuration mode

☒ **Basic**
Define a data protection policy using a simple menu.

☐ **Advanced**
Define a custom data protection policy using JSON.

Refer to the Appendix for the code

Select the FacultyArletsTopic

aws Search [Alt+S] Asia Pacific (Singapore) Account ID: 0196-6205-9667 calvin

Amazon SNS > Topics

Amazon SNS

- Dashboard
- Topics
- Subscriptions

▼ **Mobile**

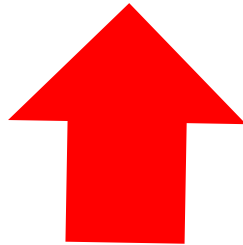
- Push notifications
- Text messaging (SMS)

Topics (1)

Search

Buttons: Edit, Delete, Publish message, Create topic

	Name	Type	ARN
☐	FacultyAlertsTopic	Standard	arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic



Click Create Subscription

 **New Feature**
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#) 

FacultyAlertsTopic

EditDeletePublish message

Details

Name FacultyAlertsTopic	Display name Faculty Alerts
ARN arn:aws:sns:southwest-1:019662039667:FacultyAlertsTopic	Topic owner 019662039667
Type Standard	

[Subscriptions](#) | [Access policy](#) | [Data protection policy](#) | [Delivery policy \(HTTP/S\)](#) | [Delivery status logging](#) | [Encryption](#) | [Tags](#) | [Integrations](#)

Subscriptions (1)

EditDeleteRequest confirmationConfirm subscriptionCreate subscription

< 1 > 

ID	Endpoint	Status	Protocol
 cf089935e-7637-4973-ae26-3033c6f0bd02	calvintanqingheng@gmail.com	 Confirmed	EMAIL

Insert Topic ARN, Select 'Email' Protocol, Insert Subscription Email, Click the 'Create Subscription' Button

Create subscription

Details

Topic ARN

Protocol
The type of endpoint to subscribe to.

Endpoint
An email address that can receive notifications from Amazon SNS.

① After your subscription is created, you must confirm it. [info](#)

► **Subscription filter policy - optional** [info](#)
This policy filters the messages that a subscriber receives.

► **Redrive policy (dead-letter queue) - optional** [info](#)
Send undeliverable messages to a dead-letter queue.

[Cancel](#) [Create subscription](#)

Open the Email to confirm subscription

AWS Notification - Subscription Confirmation Inbox x



Faculty Alerts

to me ▼

11:06 PM (7 minutes ago)

You have chosen to subscribe to the topic:

arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):

[Confirm subscription](#)



Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)

↩ Reply

➦ Forward



Subscription confirmation message



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

`arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic:cf08993a-7637-4972-ae26-5033cbf0bd02`

If it was not your intention to subscribe, [click here to unsubscribe](#).

Test on the Simple Notification Service

Back to lambda Function, select SendFaculty Alerts

Functions (7)

Last fetched 10 minutes ago  Actions 

 Filter by attributes or search by keyword

☐	Function name	Description	Package type	Runtime	Last modified
☐	dynamoDBtoS3Exporter	-	Zip	Node.js 22.x	3 weeks ago
☐	updateS3	-	Zip	Python 3.13	3 weeks ago
☐	getActivity	-	Zip	Python 3.13	3 weeks ago
☐	updateClassrom	-	Zip	Node.js 22.x	14 hours ago
☐	dashboard_api	-	Zip	Node.js 22.x	14 hours ago
☐	ClassroomDataFetcher	-	Zip	Python 3.13	3 days ago
☐	SendFacultyAlerts	-	Zip	Node.js 22.x	20 hours ago



Select test

SendFacultyAlerts Throttle Copy ARN Actions

▼ **Function overview** [Info](#)

Diagram | Template

 **SendFacultyAlerts**
Layers (0)

 **DynamoDB** (2)  **SNS** (0)

+ Add trigger + Add destination

Description
-

Last modified
20 hours ago

Function ARN
 `arn:aws:lambda:ap-southeast-1:019662059667:function:SendFacultyAlerts`

Function URL [Info](#)
-

[Export to Infrastructure Composer](#) Download

[Code](#) | [Test](#) | [Monitor](#) | [Configuration](#) | [Aliases](#) | [Versions](#)



Create new Test, insert the necessary code, save and test it

Code

Test

Monitor

Configuration

Aliases

Versions

Test event Info

To invoke your function without saving an event, configure the JSON event, then choose Test.

CloudWatch Logs Live Tail

Save

Test

Test event action

☒ Create new event

☐ Edit saved event

Event name

TestSNS

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private

☐ Shareable

Template - optional

test2

Event JSON

Format JSON

```
1 * [
2 *   "Records": [
3 *     {
4 *       "eventID": "1",
5 *       "eventName": "MODIFY",
6 *       "eventVersion": "1.1",
7 *       "eventSource": "aws:dynamodb",
8 *       "awsRegion": "ap-southeast-1",
9 *       "dynamodb": {
10 *         "ApproximateCreationDateTime": 1636470000,
11 *         "Keys": {
12 *           "classId": { "S": "N001" }
13 *         },
14 *         "NewImage": {
15 *           "classId": { "S": "N001" },
16 *           "maximumCapacity": { "N": "25" },
17 *           "currentCount": { "N": "27" },
18 *           "timestamp": { "S": "10/09/2025 15:00:59" }
19 *         },
20 *         "SequenceNumber": "222",
21 *         "SizeBytes": 50,
22 *         "StreamViewType": "NEW_AND_OLD_IMAGES"
23 *       },
24 *       "eventSourceARN": "arn:aws:dynamodb:ap-southeast-1:019662059667:table/classroom_classroom/stream/2025-09-10T00:00:00.000"
25 *     }
26 *   ]
27 * }
28
```

1:1 JSON

Check the Test response of SNS in Log events

CloudWatch interface showing the Log group details for `/aws/lambda/SendFacultyAlerts`.

The top navigation bar includes tabs: Code, Test, Monitor, Configuration, Aliases, Versions.

A green banner at the top indicates: **Executing function: succeeded (logs [2])** with a **Details** link.

The left sidebar shows the CloudWatch navigation menu with sections: Dashboards, AI Operations, Alarms, Logs, Metrics, Application Signals (APM), Network Monitoring, and Insights.

The main content area displays the **Log group details** for `/aws/lambda/SendFacultyAlerts`. The details include:

- Log class:** Standard
- ARN:** `arn:aws:logs:ap-southeast-1:019662059667:log-group:/aws/lambda/SendFacultyAlerts:`
- Creation time:** 19 days ago
- Retention:** Never expire
- Stored bytes:** 8,09 KB
- Metric filters:** 0
- Subscription filters:** 0
- Contributor Insights rules:** -
- KMS key ID:** -
- Anomaly detection:** -
- Data protection:** -
- Sensitive data count:** -
- Custom field indexes:** [Configure](#)
- Transformer:** -

Below the details, there are tabs for **Log streams**, Tags, Anomaly detection, Metric filters, Subscription filters, Contributor Insights, Data protection, Field indexes, and Transformer.

The **Log streams (22)** section shows a list of log streams. A red arrow points to the first log stream entry:

Log stream	Last event time
2025/09/10/[SLATEST]087995b7fa4b1440c9ae659c9285be220	2025-09-11 04:53:39 (UTC)

At the top right of the Log streams section, there are buttons: [Delete](#), [Create log stream](#), and [Search all log streams](#).

Check the Test response of SNS in Log events

CloudWatch > Log groups > /aws/lambda/SendFacultyAlerts > 2025/09/10/[SLATEST]08795b7fa4b1440c9ae659c9285be220

CloudWatch

Favorites and recents

Dashboards

► **AI Operations** [New](#)

► **Alarms**

▼ **Logs**

[Log groups](#)

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

► **Metrics**

► **Application Signals (APM)**

► **Network Monitoring**

► **Insights**

Settings

Telemetry config

Getting Started

What's new

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Timestamp	Message
No older events at this moment. Retry	
2025-09-11T04:33:39.036Z	INIT_START Runtime Version: nodejs:22.v53 Runtime Version ARN: arn:aws:lambda:ap-southeast-1::runtime:95d75816ec5ca62f98d61443748491f631a78c967b17393a9c60b763a4cb014
2025-09-11T04:33:39.428Z	INIT_START Runtime Version: nodejs:22.v53 Runtime Version ARN: arn:aws:lambda:ap-southeast-1::runtime:95d75816ec5ca62f98d61443748491f631a78c967b17393a9c60b763a4cb014
2025-09-11T04:33:39.428Z	START RequestId: 80d4c793-87da-4cef-9b51-39f6aa5e3b90 Version: \$LATEST
2025-09-11T04:33:39.430Z	START RequestId: 80d4c793-87da-4cef-9b51-39f6aa5e3b90 Version: \$LATEST
2025-09-11T04:33:39.430Z	2025-09-11T04:33:39.430Z 80d4c793-87da-4cef-9b51-39f6aa5e3b90 INFO Publishing OVERLOAD message to SNS: Classroom ID: N001 Full capacity: 25 Current capacity: 27 Status: The class is overloaded Timestamp: 10/09/2025 15:00:59
2025-09-11T04:33:39.430Z	80d4c793-87da-4cef-9b51-39f6aa5e3b90 INFO Publishing OVERLOAD message to SNS: Classroom ID: N001 Full capacity: 25 Current capacity: 27 Status: The class is overloaded Timestamp: 10/09/2025 15:00:59
2025-09-11T04:33:40.436Z	END RequestId: 80d4c793-87da-4cef-9b51-39f6aa5e3b90
2025-09-11T04:33:40.436Z	END RequestId: 80d4c793-87da-4cef-9b51-39f6aa5e3b90
2025-09-11T04:33:40.436Z	REPORT RequestId: 80d4c793-87da-4cef-9b51-39f6aa5e3b90 Duration: 990.27 ms Billed Duration: 1379 ms Memory Size: 128 MB Max Memory Used: 95 MB Init Duration: 388.20 ms
2025-09-11T04:33:40.436Z	REPORT RequestId: 80d4c793-87da-4cef-9b51-39f6aa5e3b90 Duration: 990.27 ms Billed Duration: 1379 ms Memory Size: 128 MB Max Memory Used: 95 MB Init Duration: 388.20 ms
No newer events at this moment. Auto retry paused. Resume	

Check on email for the SNS Notification

 Faculty Monitoring Alert: Overloaded Classroom Inbox x



Faculty Alerts

to me ▾

12:33 PM (12 minutes ago)



Classroom ID: N001

Full capacity: 25

Current capacity: 27

Status: The class is overloaded

Timestamp: 10/09/2025 15:00:59

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:

<https://sns.ap-southeast-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic:cf08993a-7637-4972-ae26-5033cbf0bd02&Endpoint=calvintanjinghang@gmail.com>

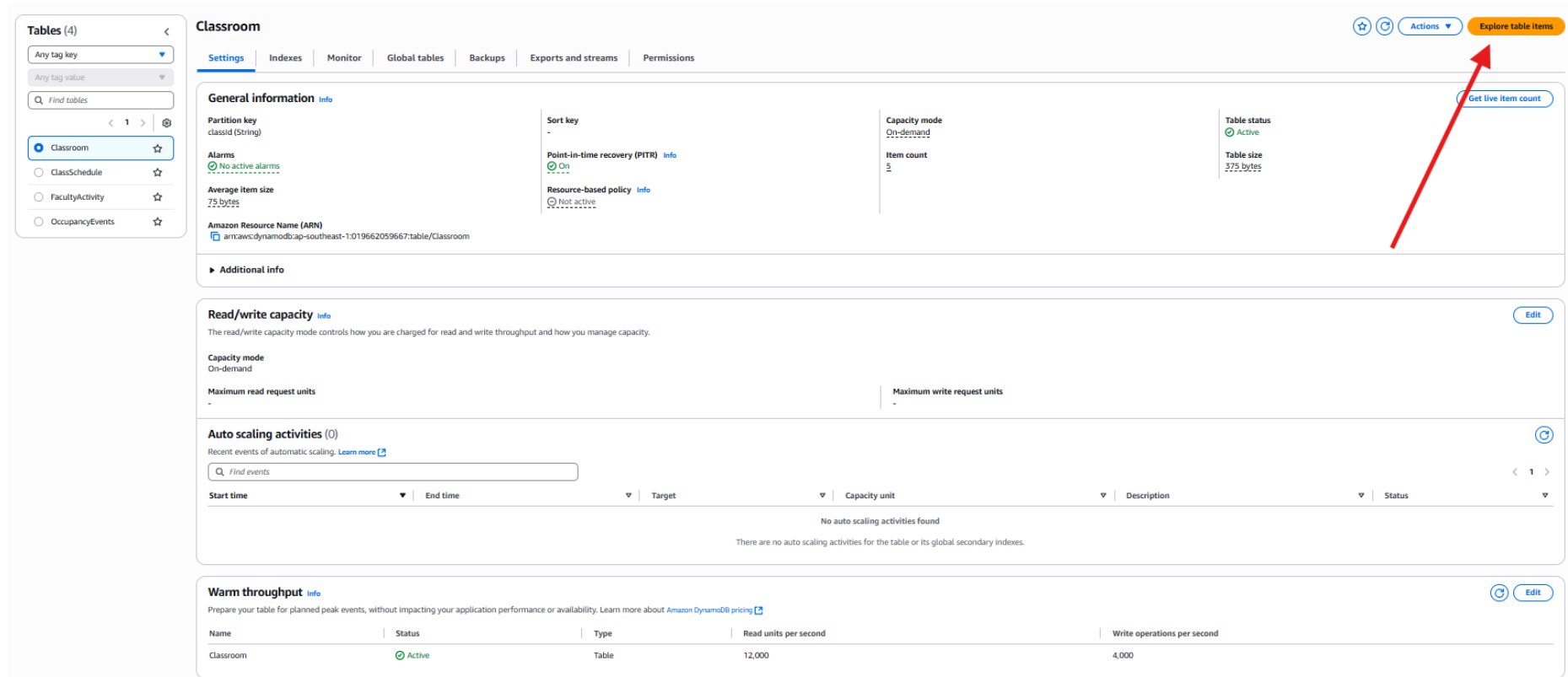
Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

Navigate to DynamoDB, Table, then select 'Classroom' table

The screenshot shows the AWS DynamoDB console interface. On the left is a navigation sidebar with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main area is titled 'Tables (4)' and contains a search bar and a table listing four tables: Classroom, ClassSchedule, FacultyActivity, and OccupancyEvents. A red arrow points to the 'Classroom' table in the list.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size	Table class
Classroom	Active	classid (S)	-		0 0	Off	☆	On-demand	On-demand	375 bytes	Standard
ClassSchedule	Active	classid (S)	courseName (S)		0 0	Off	☆	On-demand	On-demand	497 bytes	Standard
FacultyActivity	Active	room_id (S)	timestamp (S)		0 0	Off	☆	On-demand	On-demand	43 bytes	Standard
OccupancyEvents	Active	classid (S)	timestamp (S)		0 0	Off	☆	On-demand	On-demand	3.4 kilobytes	Standard

In the Classroom Table, Select 'Explore Table Item'



The screenshot shows the Amazon DynamoDB console for the 'Classroom' table. On the left, a sidebar lists tables: Classroom (selected), ClassSchedule, FacultyActivity, and OccupancyEvents. The main content area has tabs for Settings, Indexes, Monitor, Global tables, Backups, Exports and streams, and Permissions. The 'Settings' tab is active, showing general information, read/write capacity, auto scaling activities, and warm throughput.

General information

- Partition key: classid (String)
- Sort key: -
- Capacity mode: On-demand
- Item count: 5
- Table status: Active
- Table size: 375 bytes
- Alarms: No active alarms
- Point-in-time recovery (PITR): On
- Resource-based policy: Not active
- Average item size: 75 bytes
- Amazon Resource Name (ARN): arn:aws:dynamodb:southeast-1:019662059667:table/Classroom

Read/write capacity

The read/write capacity mode controls how you are charged for read and write throughput and how you manage capacity.

Capacity mode: On-demand

Maximum read request units: -

Maximum write request units: -

Auto scaling activities (0)

Recent events of automatic scaling. [Learn more](#)

[Find events](#)

Start time	End time	Target	Capacity unit	Description	Status
No auto scaling activities found					

There are no auto scaling activities for the table or its global secondary indexes.

Warm throughput

Prepare your table for planned peak events, without impacting your application performance or availability. [Learn more about Amazon DynamoDB pricing](#)

Name	Status	Type	Read units per second	Write operations per second
Classroom	Active	Table	12,000	4,000

A red arrow points to the 'Explore table items' button in the top right corner of the console.

Modify the currentlyCount to exceed the maximumCapacity

The screenshot shows a web interface for managing data. On the left, a sidebar lists tables: Classroom, ClassSchedule, FacultyActivity, and OccupancyEvents. The main area is titled 'Classroom' and shows a 'Scan or query items' section. Below this, a status bar indicates 'Completed - Items returned: 5 - Items scanned: 5 - Efficiency: 100% - RCUs consumed: 2'. The table 'Classroom - Items returned (5)' is displayed, with columns: classId (String), currentCount, maximumCapac..., and timestamp. A red box highlights the table data. An 'Edit Number' dialog box is open, showing the 'currentCount' for item N003 being changed from 4 to 54. A red arrow points to the 'Save' button in the dialog.

Tables (4)

- Any tag key
- Any tag value
- Find tables
- Classroom
- ClassSchedule
- FacultyActivity
- OccupancyEvents

Classroom

Autopreview View table details

▼ Scan or query items

Scan Query

Select a table or index

Table - Classroom

Select attribute projection

All attributes

Filters - optional

Run Reset

Completed - Items returned: 5 - Items scanned: 5 - Efficiency: 100% - RCUs consumed: 2

Table: Classroom - Items returned (5)

Scan started on September 11, 2025, 12:42:00

Actions Create Item

	classId (String)	currentCount	maximumCapac...	timestamp
☐	N001	26	25	08/09/2025, 14:57:58.526
☐	N002	50	50	08/09/2025, 14:05:12.438
☐	N004	6	50	08/09/2025, 14:05:10.774
☐	N005	2	50	08/09/2025, 14:05:09.271
☐	N003	4		08/09/2025, 14:05:11.848

Edit Number

54

Enter any number up to 38 digits. Leading and trailing zeros are ignored.

Cancel Save

Check on email for the SNS Notification



Faculty Alerts

to me ▾

12:45 PM (0 minutes ago)



Classroom ID: N003

Full capacity: 50

Current capacity: 54

Status: The class is overloaded

Timestamp: 08/09/2025, 14:05:11.848



↩ Reply

➦ Forward



Check the Test response of SNS in Log events

/aws/lambda/SendFacultyAlerts Actions View in Logs Insights Start tailing Search log group

▼ Log group details

Log class [Info](#)
Standard

ARN
[arn:aws:logs:ap-southeast-1:019662059667:log-group:/aws/lambda/SendFacultyAlerts:](#)

Creation time
20 days ago

Retention
Never expire

Stored bytes
6.05 KB

Metric filters
0

Subscription filters
0

Contributor Insights rules
-

KMS key ID
-

Anomaly detection
-

Data protection
-

Sensitive data count
-

Custom field indexes
[Configure](#)

Transformer
ERROR

[Log streams](#) | [Tags](#) | [Anomaly detection](#) | [Metric filters](#) | [Subscription filters](#) | [Contributor Insights](#) | [Data protection](#) | [Field indexes](#) | [Transformer](#)

Log streams (23) 🔄 Delete Create log stream Search all log streams

filter log streams or try prefix search ☐ Exact match ☐ Show expired [Info](#)

☐ Log stream	▼ Last event time
☐ 2025/09/10/[SLATEST]9884a6dbdc9f402aa3f30fb254662a6e	2025-09-11 04:45:42 (UTC)
☐ 2025/09/10/[SLATEST]08795b7fa4b1440c3ae659c9285be220	2025-09-11 04:33:39 (UTC)

Check the Test response of SNS in Log events

The screenshot shows the AWS CloudWatch console interface. The left sidebar contains navigation links for CloudWatch, Dashboards, Alarms, Logs, Metrics, Application Signals (APM), Network Monitoring, and Insights. The main panel displays the 'Log events' for the Lambda function `/aws/lambda/SendFacultyAlerts`. The log entries are as follows:

Timestamp	Message
2025-09-11T04:45:41.428Z	INIT_START Runtime Version: nodejs12.v53 Runtime Version ARN: arn:aws:lambda:ap-southeast-1::runtime:95d7883e6ca62f98d865443748491f631a78c967b17393akc6b763akc8014
2025-09-11T04:45:41.430Z	START RequestId: b01e4873-7940-4576-b42c-2642f7739609 Version: SLATEST
2025-09-11T04:45:41.432Z	2025-09-11T04:45:41.432Z b01e4873-7940-4576-b42c-2642f7739609 INFO Publishing OVERLOAD message to SNS: Classroom ID: N003 Full capacity: 50 Current capacity: 54 Status: The class is overloaded timestamp: 09/09/2025, 14:05:11.848
2025-09-11T04:45:42.350Z	END RequestId: b01e4873-7940-4576-b42c-2642f7739609
2025-09-11T04:45:42.350Z	REPORT RequestId: b01e4873-7940-4576-b42c-2642f7739609 Duration: 929.77 ms Billed Duration: 1338 ms Memory Size: 128 MB Max Memory Used: 97 MB Init Duration: 387.93 ms

A red box highlights the log entries from the first 'START' event to the last 'REPORT' event. A red arrow points to the message 'Status: The class is overloaded' in the log entry at timestamp 2025-09-11T04:45:41.432Z.

Dashboards

Navigate to the API Gateway console and create an API

Networking & Content Delivery

API Gateway

Create and manage APIs at scale

Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs.

How it works



API Gateway enables you to connect and access data, business logic, and functionality from backend services such as workloads running on Amazon Elastic Compute Cloud (Amazon EC2), code running on AWS Lambda, any web application, or real-time communication applications.

Get started

Create a new API to begin exploring API Gateway. You can also import an external definition file into API Gateway.

Create an API

Pricing

With Amazon API Gateway, you only pay when your APIs are used. There are no minimum fees or upfront commitments. For HTTP and REST APIs, you pay based on API calls received and amount of data transferred out. For WebSocket APIs, you pay based on number of messages and connection duration.

[Learn more about pricing](#)

Resources

[Getting Started Guide](#)

[Developer Guide](#)

[API References](#)

Select REST API

Choose an API type [Info](#)

HTTP API

Build low-latency and cost-effective REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.

Works with the following:
Lambda, HTTP backends

[Import](#) [Build](#)

WebSocket API

Build a WebSocket API using persistent connections for real-time use cases such as chat applications or dashboards.

Works with the following:
Lambda, HTTP, AWS Services

[Import](#) [Build](#)

REST API

Develop a REST API where you gain complete control over the request and response along with API management capabilities.

Works with the following:
Lambda, HTTP, AWS Services

[Import](#) [Build](#)



Give the API a name and a description

Create REST API [Info](#)

API details

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name

classroomDataAPI

Description - optional

Retrieve data for the analytics dashboard

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.

Regional

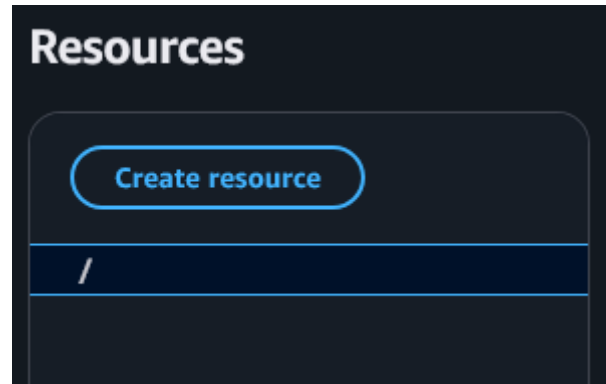
IP address type [Info](#)
Select the type of IP addresses that can invoke the default endpoint for your API.

☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.

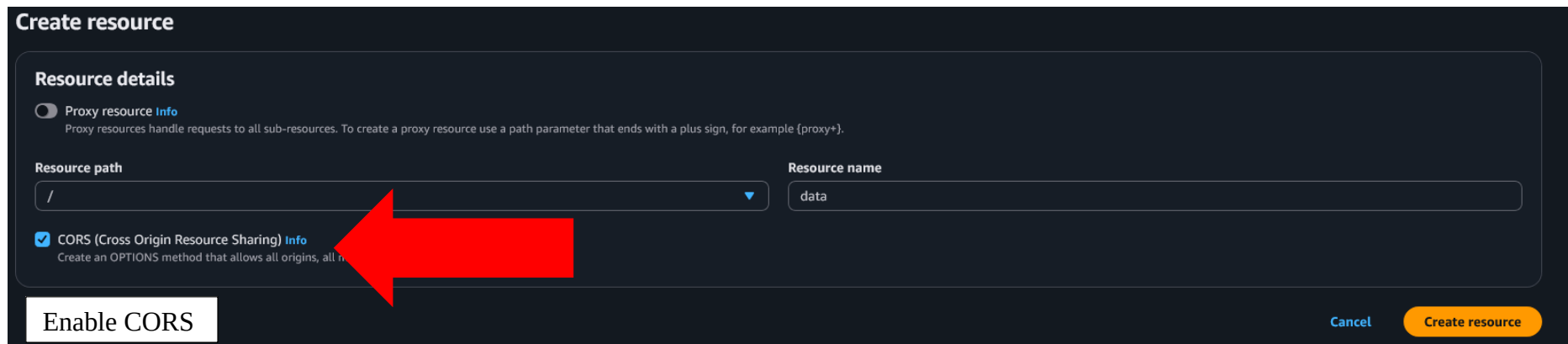
☐ **Dualstack**
Supports all API endpoint types.

[Cancel](#) [Create API](#)

Navigate to the created API and press create resource



Name the resource and create resource



The screenshot displays the 'Create resource' form. Under the 'Resource details' section, the 'Proxy resource' toggle is off. The 'Resource path' dropdown menu is set to '/'. The 'Resource name' text input contains 'data'. The 'CORS (Cross Origin Resource Sharing)' checkbox is checked. A red arrow points to the 'Resource path' field. At the bottom, there is a white button labeled 'Enable CORS', a blue 'Cancel' link, and an orange 'Create resource' button.

Create a method

Methods (1)

Delete

Create method

	Method type	Integration type	Authorization	API key
☐	OPTIONS	Mock	None	Not required



Select GET method type, choose the lambda function that queries the data with AWS Athena, and enable lambda proxy integration

Method details

Method type

GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **Mock**
Generate a response based on API Gateway mappings and transformations.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.

☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Lambda function
Provide the Lambda function name or alias. You can also provide an ARN from another account.

ap-southeast-1

arn:aws:lambda:ap-southeast-1:01966...:function:d...

Grant API Gateway permission to invoke your Lambda function
When you save your changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.

Integration timeout | [Info](#)
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms.

29000

Refer to the appendix for examples of code

1. ClassroomDataFetcher
2. fetchHistoricalData

Deploy the API

Deploy API

Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage

New stage

Select new stage or use an existing one

Stage name

test

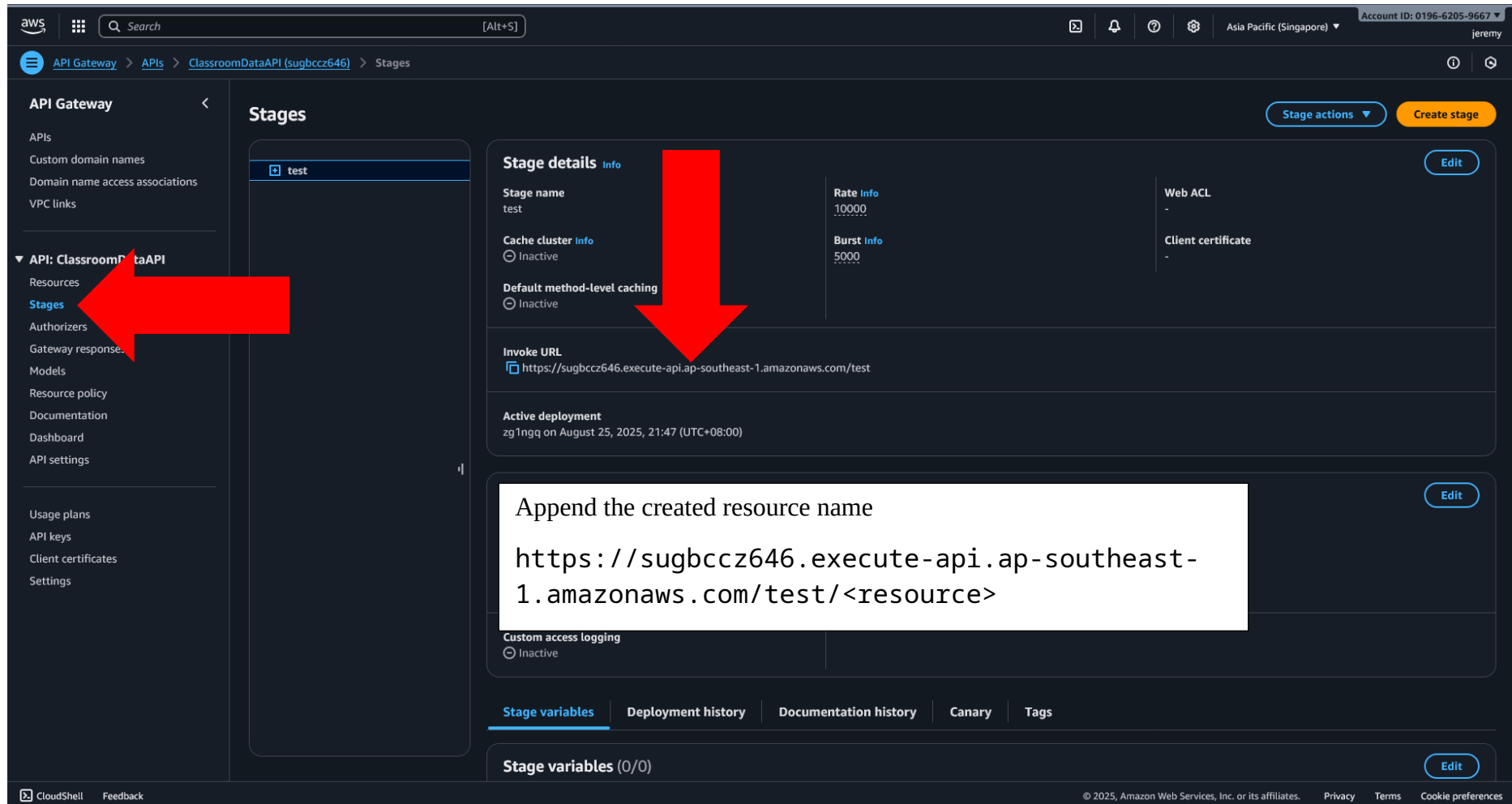
i A new stage will be created with the default settings. Edit your stage settings on the **Stage** page.

Deployment description

Cancel

Deploy

Copy the invoke URL from the Stages section



The screenshot displays the AWS API Gateway console interface. On the left, a navigation sidebar lists various API Gateway components, with 'API: ClassroomDataAPI' expanded and 'Stages' selected. A large red arrow points from the 'Stages' link in the sidebar to the 'test' stage in the main content area. The 'test' stage is highlighted in the 'Stages' list. The main content area shows the 'Stage details' for the 'test' stage. A second large red arrow points from the 'Invoke URL' field to a text box. The 'Invoke URL' field contains the URL: `https://sugbccz646.execute-api.ap-southeast-1.amazonaws.com/test`. The text box contains the instruction: 'Append the created resource name' followed by the URL template: `https://sugbccz646.execute-api.ap-southeast-1.amazonaws.com/test/<resource>`. The 'Stage details' section also includes fields for 'Stage name' (test), 'Rate' (10000), 'Burst' (5000), 'Cache cluster' (Inactive), 'Default method-level caching' (Inactive), 'Web ACL' (-), and 'Client certificate' (-). The 'Active deployment' section shows the deployment 'zg1ngq' on August 25, 2025, at 21:47 (UTC+08:00). The 'Custom access logging' section is also visible, showing it is inactive. At the bottom, there are tabs for 'Stage variables', 'Deployment history', 'Documentation history', 'Canary', and 'Tags'. The 'Stage variables' tab is active, showing 'Stage variables (0/0)'. The footer of the console includes 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc. or its affiliates.

API Gateway

- APIs
- Custom domain names
- Domain name access associations
- VPC links
- API: ClassroomDataAPI
 - Resources
 - Stages
 - Authorizers
 - Gateway response
 - Models
 - Resource policy
 - Documentation
 - Dashboard
 - API settings
- Usage plans
- API keys
- Client certificates
- Settings

Stages

test

Stage details

Stage name: test

Rate: 10000

Burst: 5000

Cache cluster: Inactive

Default method-level caching: Inactive

Web ACL: -

Client certificate: -

Invoke URL: `https://sugbccz646.execute-api.ap-southeast-1.amazonaws.com/test`

Active deployment: zg1ngq on August 25, 2025, 21:47 (UTC+08:00)

Custom access logging: Inactive

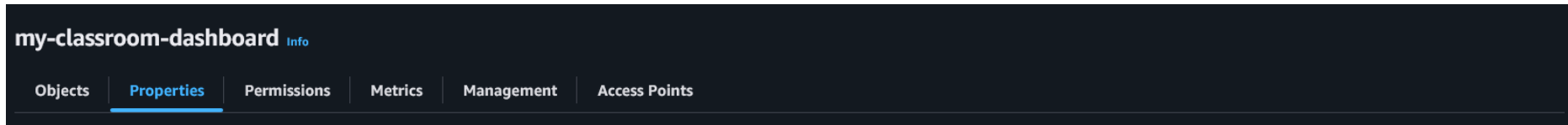
Append the created resource name

`https://sugbccz646.execute-api.ap-southeast-1.amazonaws.com/test/<resource>`

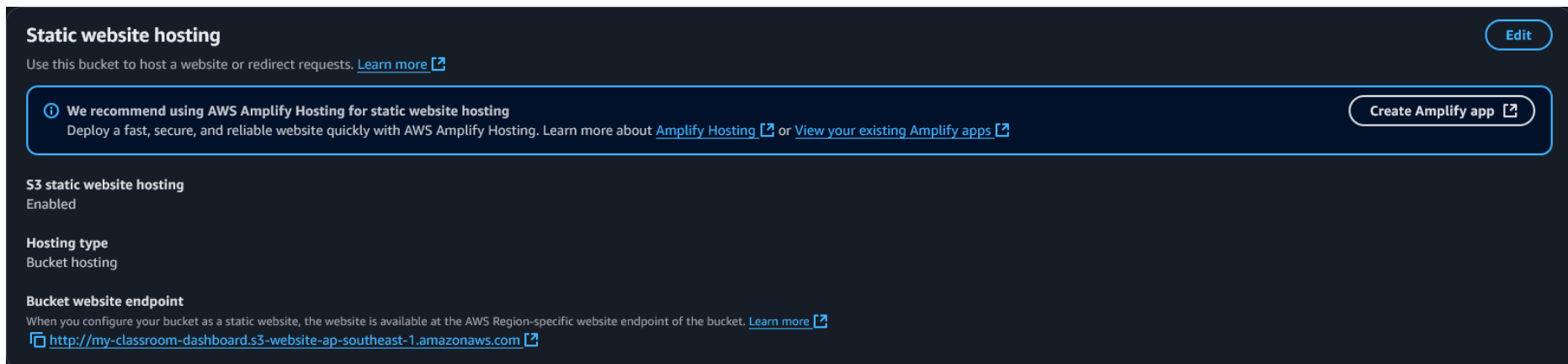
Stage variables (0/0)

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

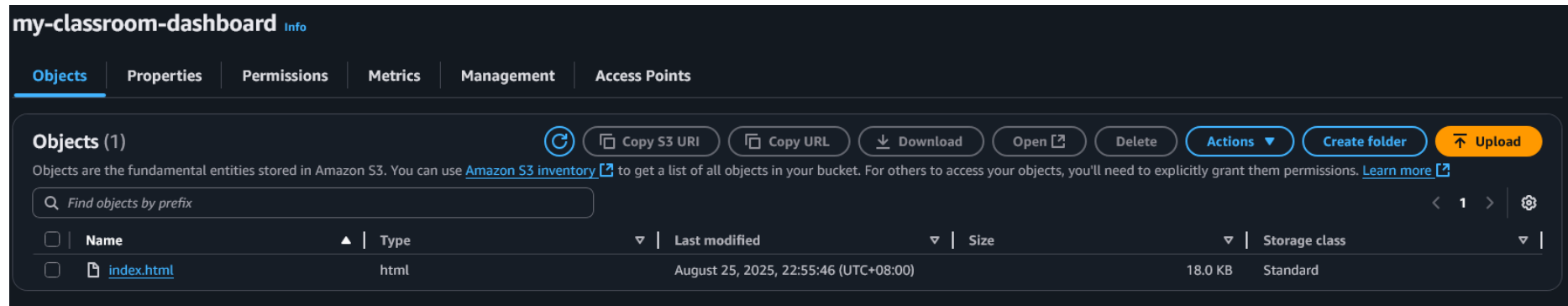
Create a bucket to host the static website



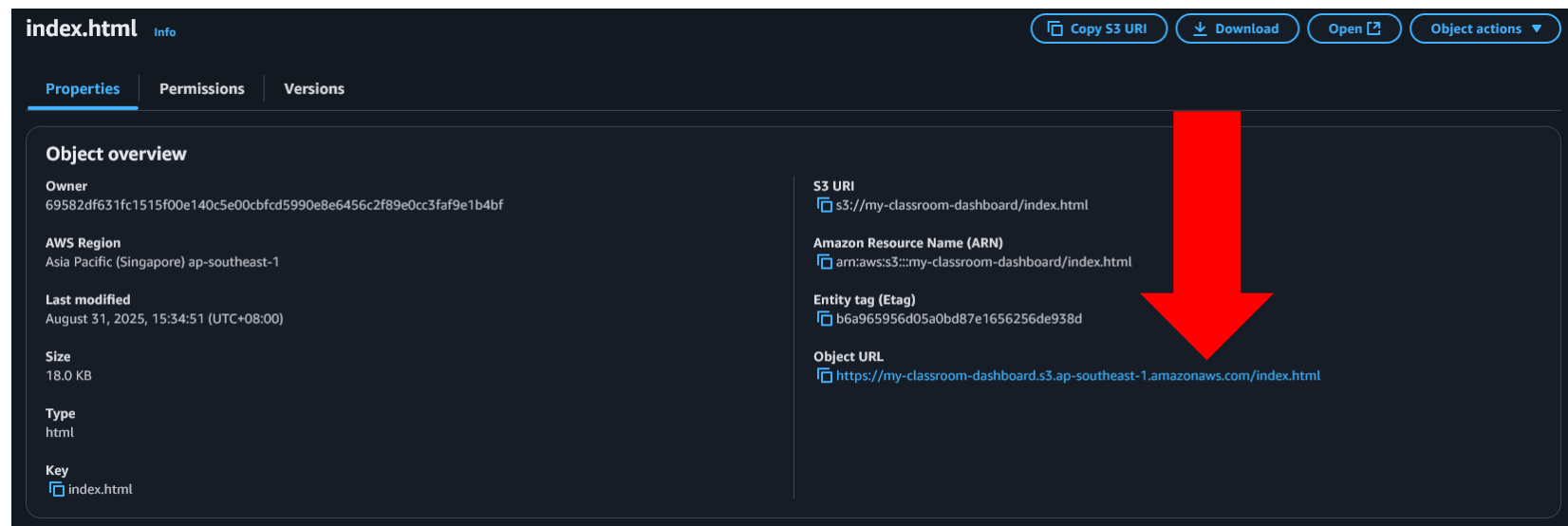
Enable s3 static website hosting



Upload the dashboard HTML code



View the dashboard with the object link provided



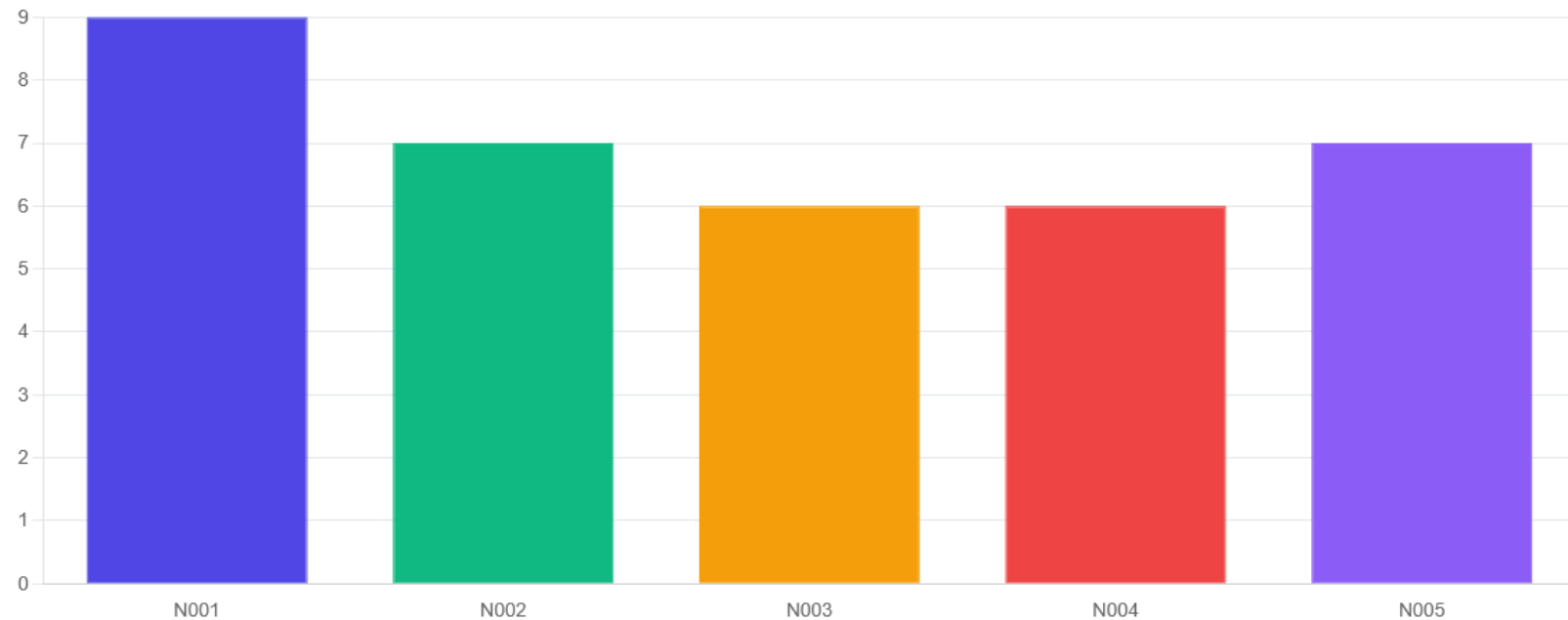
Dashboard Live View Example 1

Classroom Occupancy

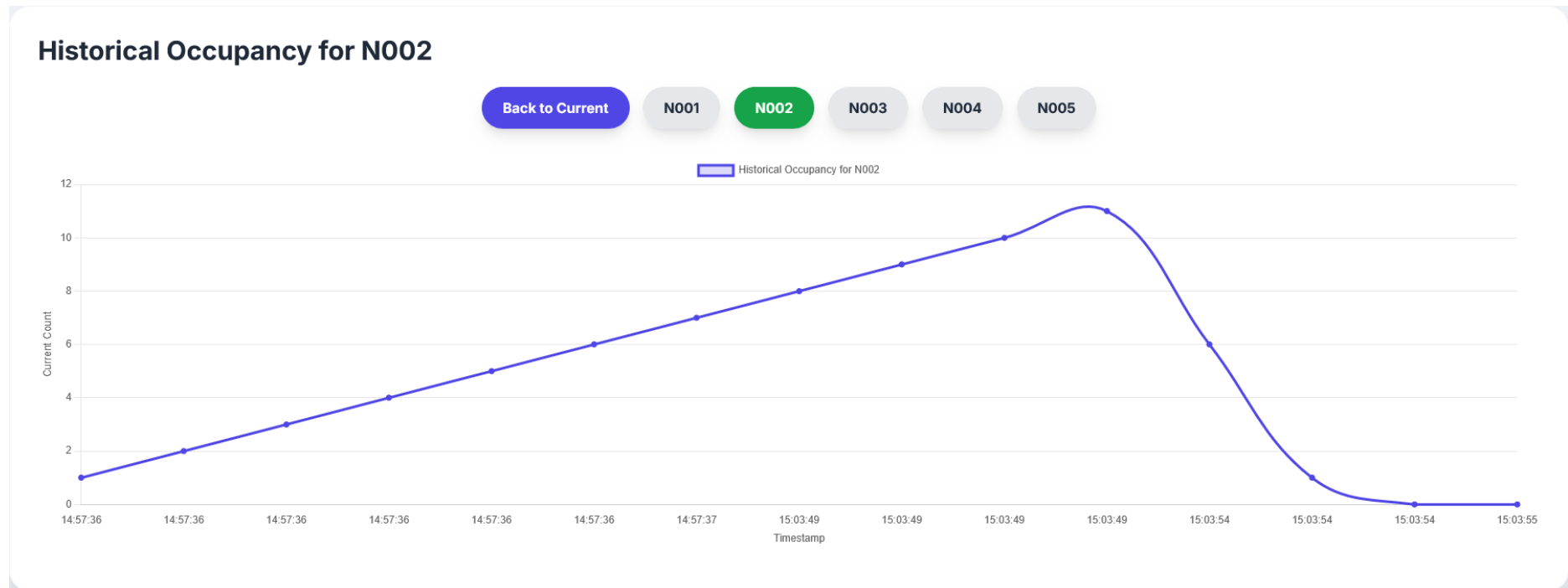
Current Data

Refresh Data

Historical Data



Dashboard Live View Example 2



Dashboard Live View Example 3

Classroom Status Overview

CLASS ID	COUNT	MAX CAPACITY	STATUS	ALERT LIGHT	SUGGESTION
N001	50	25	OVERLOAD (200%): Capacity exceeded!		ALERT MODERATOR! Suggest another classroom with suitable capacity.
N002	0	50	Unoccupied (0%): Classroom is empty!		Switching off the lights and air-conditioner
N003	0	50	Unoccupied (0%): Classroom is empty!		Switching off the lights and air-conditioner
N004	0	50	Unoccupied (0%): Classroom is empty!		Switching off the lights and air-conditioner
N005	0	50	Unoccupied (0%): Classroom is empty!		Switching off the lights and air-conditioner

Activity Monitoring System

Repeat above process with different lambda name and file for S3 to create another static website



Appendix

Reset Flow Context

```
// Get the class ID from the message
let classId = msg.classId || "ClassA";

// Initialize flow.classData if it doesn't exist
let classData = flow.get('classData') || {};
if (!classData[classId]) {
    classData[classId] = { occupancy: 0 };
}

classData[classId].occupancy = 0;
classData[classId].actualCount = undefined;

flow.set('classData', classData); // Save the updated classData object
msg.payload = 0;
msg.topic = 'reset_trigger';
msg.classId = classId;
return msg;
```

Occupancy Counter

```
// Initialize classroomOccupancy in flow context if it doesn't exist
let classId = msg.classId || 'ClassA';

let classData = flow.get('classData') || {};
if (!classData[classId]) {
    classData[classId] = {
        occupancy: 0,
        actualCount: 0,
        capacity: 50
    };
}

let occupancy = classData[classId].occupancy;
var action

// Check if the incoming message noOfStudent is a number (1 for entry, -1
for exit)
if (typeof msg.noOfStudent === 'number') {
    occupancy += msg.noOfStudent;
}

// Ensure occupancy never goes below zero (you can't have negative students)
if (occupancy < 0) {
    occupancy = 0;
}

if (msg.noOfStudent > 0 ) {
    action = "Enter"
}
else{
    action = "Exit"
}
```

```
    "Name": msg.payload.Name,  
    "action": action,  
    "classID": classId,  
    "occupancy": msg.curTotalNumber  
};  
  
// Assign the newly created object to the message payload  
msg.payload = newPayload;  
  
return msg;
```

2 Msg modifier

```
var timestamp = new Date().toLocaleString("en-MY", {
  timeZone: "Asia/Kuala_Lumpur",
  year: 'numeric',
  month: '2-digit',
  day: '2-digit',
  hour: '2-digit',
  minute: '2-digit',
  second: '2-digit',
  hour12: false
}) + '.' + new Date().getMilliseconds().toString().padStart(3, '0');

var classid = msg.classId
var occupancy = msg.payload.occupancy

// Create a new JavaScript object with the desired keys
var newPayload = {
  "Name": msg.payload.Name,
  "type": "classroom_update",
  "classID": classid,
  "occupancy": occupancy,
  "timestamp": timestamp,
};

msg.payload = newPayload;

return msg;
```

```
var timestamp = new Date().toLocaleString("en-MY", {
  timeZone: "Asia/Kuala_Lumpur",
  year: 'numeric',
```

```
// Create a new JavaScript object with the desired keys
var newPayload = {
  "Name": msg.payload.Name,
  "type": "occupancy_event",
  "action": msg.payload.action,
  "classID": classid,
  "timestamp": timestamp,
};

msg.payload = newPayload;

return msg;
```

updateClassroom

```
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import {
  DynamoDBDocumentClient,
  UpdateCommand,
  PutCommand
} from "@aws-sdk/lib-dynamodb";

const client = new DynamoDBClient({});
const db = DynamoDBDocumentClient.from(client);

export const handler = async (event) => {
  const type = event.type;
  const classId = event.classID || event.classId;

  if (!type || !classId) {
    return {
      statusCode: 400,
      body: JSON.stringify('Missing "type" or "classId/classID" in the
event.')}
    };
  }

  try {
    switch (type) {
      case 'classroom_update':
        const {
          currentCount,
          timestamp: providedTimestamp
        } = event.data;

        if (currentCount === undefined) {
          console.error('Validation Error: Missing "currentCount"
in the event data.');
```



```

        Key: {
            'classId': classId
        },
        UpdateExpression: 'SET currentCount = :c, #ts = :t',
        ExpressionAttributeNames: {
            '#ts': 'timestamp'
        },
        ExpressionAttributeValues: {
            ':c': currentCount,
            ':t': timestamp
        },
        ReturnValues: 'UPDATED_NEW'
    };

    await db.send(new UpdateCommand(classroomUpdateParams));
    return {
        statusCode: 200,
        body: JSON.stringify(`Classroom ${classId} updated
successfully.`)
    };

    case 'occupancy_event':
        const { action } = event.data;
        const eventTimestamp = event.data.timestamp;

        if (!action || !eventTimestamp) {
            console.error('Validation Error: Missing "action" or
"timestamp" in the event.');
```

```

            return {
                statusCode: 400,
                body: JSON.stringify('Missing "action" or
"timestamp" in the event.')
```

```

            };
        }

        const occupancyEventParams = {
            TableName: 'OccupancyEvents'
        };

```

```

        body: JSON.stringify(`Occupancy event for classroom
${classId} recorded successfully.`)
    };

    case 'schedule_update':
        const {
            courseName,
            scheduleType,
            dayOfWeek,
            startTime,
            endTime
        } = event.data;

        if (!courseName || !scheduleType || !dayOfWeek || !startTime
|| !endTime) {
            console.error('Validation Error: Missing required fields
in the event.');
```

event.')

```

            return {
                statusCode: 400,
                body: JSON.stringify('Missing required fields in the
event.')
```

};

```

        }

        const scheduleUpdateParams = {
            TableName: 'ClassSchedule',
            Key: {
                'classId': classId,
                'courseName': courseName,
            },
            UpdateExpression: 'SET #type = :t, dayOfWeek = :d,
startTime = :st, endTime = :et',
            ExpressionAttributeNames: {
                '#type': 'type'
            },
            ExpressionAttributeValues: {
                ':t': scheduleType

```

```

        body: JSON.stringify(`Schedule for classroom ${classId}
updated successfully.`)
    };

    default:
        return {
            statusCode: 400,
            body: JSON.stringify(`Unknown event type: ${type}`)
        };
    }
} catch(error) {
    return {
        statusCode: 500,
        body: JSON.stringify(`DynamoDB Operation Error for type
"${type}": ${error.message}`)
    };
}
};

```

store_Classroom_Data

```

SELECT type AS type,
       classID AS classId,
       { "currentCount": occupancy } AS data
FROM 'IRsensor_Class/node-red'
WHERE type = 'classroom_update'

```

store_OccupancyEvent_Data

```

SELECT type AS type

```

DynamoDBtoS3Exporter

```
import {
  S3Client,
  PutObjectCommand
} from "@aws-sdk/client-s3";

import {
  unmarshall
} from "@aws-sdk/util-dynamodb";

const s3 = new S3Client({});
const BUCKET_NAME = 'utility-classroom-data-lake';

export const handler = async (event) => {
  for (const record of event.Records) {
    if (record.eventName === 'INSERT' || record.eventName === 'MODIFY')
    {
      const newImage = record.dynamodb.NewImage;
      const recordData = unmarshall(newImage);
      const tableName = record.eventSourceARN.split('/')[1];
      const s3Key =
`${tableName}/${recordData.classId}/${Date.now()}.json`;
      const params = {
        Bucket: BUCKET_NAME,
        Key: s3Key,
        Body: JSON.stringify(recordData),
        ContentType: 'application/json'
      };

      try {
        const command = new PutObjectCommand(params);
        await s3.send(command);
        console.log(`Successfully uploaded to S3:
s3://${BUCKET_NAME}/${s3Key}`);
      } catch (error) {
```

ClassroomDataFetcher

```
import boto3
import time
import json

def lambda_handler(event, context):
    athena_client = boto3.client('athena')
    s3_bucket = 'utility-classroom-data-lake'
    athena_database = 'classroom_analytics'
    athena_table = 'classroom_classroom'

    sql_query = f"""
        WITH ranked_data AS (
            SELECT
                classId,
                currentCount,
                maximumCapacity,
                ROW_NUMBER() OVER (
                    PARTITION BY classId
                    ORDER BY
                        CASE
                            -- ISO 8601 format
                            WHEN timestamp LIKE '%T%' THEN
                                from_iso8601_timestamp(timestamp)
                            -- Custom format with slashes
                            WHEN timestamp LIKE '%/%' THEN
                                date_parse(timestamp, '%d/%m/%Y, %H:%i:%s.%f')
                            -- Custom format with dashes
                            WHEN timestamp LIKE '%-%' THEN
                                date_parse(timestamp, '%d-%m-%Y, %H:%i:%s.%f')
                            -- NEW: Explicitly handle and ignore the
                                malformed data format
                                WHEN timestamp LIKE '("%(%)")%' THEN NULL
                            -- Fallback for any other bad data
                            ELSE NULL
                        END
                ) AS rank
            FROM s3://{s3_bucket}/athena/{athena_table}
        )
        SELECT * FROM ranked_data
    """
```

```

        classId,
        currentCount,
        maximumCapacity
    FROM
        ranked_data
    WHERE
        rn = 1;

"""

try:
    response = athena_client.start_query_execution(
        QueryString=sql_query,
        QueryExecutionContext={'Database': athena_database},
        ResultConfiguration={'OutputLocation': f's3://{s3_bucket}/query-
results/'})

    query_execution_id = response['QueryExecutionId']

    while True:
        query_status =
athena_client.get_query_execution(QueryExecutionId=query_execution_id)
        state = query_status['QueryExecution']['Status']['State']

        if state in ['SUCCEEDED', 'FAILED', 'CANCELLED']:
            break
        time.sleep(1)

    if state == 'FAILED':
        error_message =
query_status['QueryExecution']['Status']['StateChangeReason']
        print(f"Athena query failed: {error_message}")
        return {
            'statusCode': 500,
            'body': json.dumps({'error': 'Athena query failed'}),
            'headers': {

```

```

        for row in results_response['ResultSet']['Rows'][1:]:
            data_list = [d.get('VarCharValue') for d in row['Data']]

            if len(data_list) >= 3 and all(d is not None for d in
data_list[:3]):
                try:
                    class_id = data_list[0]
                    current_count = int(data_list[1])
                    maximum_capacity = int(data_list[2])
                    results.append({'classId': class_id, 'currentCount':
current_count, 'maximumCapacity': maximum_capacity})
                except (ValueError, TypeError) as e:
                    print(f"Error parsing row: {data_list}. Error: {e}")

        return {
            'statusCode': 200,
            'body': json.dumps(results),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }

    except Exception as e:
        print(f"General error: {e}")
        return {
            'statusCode': 500,
            'body': json.dumps({'error': str(e)}),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }

# import boto3
# import time

```

```

#         classId,
#         currentCount,
#         maximumCapacity,
#         timestamp
#     FROM
#         "{athena_database}"."{athena_table}"
#     WHERE
#         timestamp IS NOT NULL
#         AND timestamp NOT LIKE '%(%' -- Exclude entries like ("Monday",
# "09:05")
#         AND timestamp NOT LIKE '%Monday%' -- Extra safety
#         AND timestamp LIKE '%/%' -- Must contain forward slash
#         AND timestamp LIKE '%, %' -- Must contain comma and space
#         AND LENGTH(timestamp) > 10 -- Must be reasonable length
#     )
#     SELECT
#         T1.classId,
#         T1.currentCount,
#         T1.maximumCapacity
#     FROM
#         valid_timestamps AS T1
#     INNER JOIN (
#         SELECT
#             classId,
#             MAX(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s.%f')) AS
latest_timestamp
#         FROM
#             valid_timestamps
#         GROUP BY
#             classId
#     ) AS T2 ON T1.classId = T2.classId
#         AND date_parse(T1.timestamp, '%d/%m/%Y, %H:%i:%s.%f') =
T2.latest_timestamp;
#     """

#     try:
#         response = athena_client.start_query_execution(

```



```

#         state = query_status['QueryExecution']['Status']['State']

#         if state in ['SUCCEEDED', 'FAILED', 'CANCELLED']:
#             break
#         time.sleep(1)

#         if state == 'FAILED':
#             error_message =
query_status['QueryExecution']['Status']['StateChangeReason']
#             return {'statusCode': 500, 'body': json.dumps({'error':
'Athena query failed'})}}

#         results_response =
athena_client.get_query_results(QueryExecutionId=query_execution_id)

#         results = []
#         if 'ResultSet' in results_response and 'Rows' in
results_response['ResultSet']:
#             for row in results_response['ResultSet']['Rows'][1:]:
#                 data_list = [d.get('VarCharValue') for d in row['Data']]

#                 if len(data_list) >= 3 and all(d is not None for d in
data_list[:3]):
#                     try:
#                         class_id = data_list[0]
#                         current_count = int(data_list[1])
#                         maximum_capacity = int(data_list[2])
#                         results.append({'classId': class_id,
'currentCount': current_count, 'maximumCapacity': maximum_capacity})
#                     except (ValueError, TypeError) as e:
#                         print(f"Error parsing row: {data_list}. Error:
{e}")

#         return {
#             'statusCode': 200,
#             'body': json.dumps(results),
#             'headers': {

```

fetchHistoricalData

```
import boto3
import time
import json
from urllib.parse import unquote

def lambda_handler(event, context):
    athena_client = boto3.client('athena')
    s3_bucket = 'utility-classroom-data-lake'
    athena_database = 'classroom_analytics'
    athena_table = 'classroom_classroom'

    try:
        query_params = event.get('queryStringParameters', {})
        class_id = query_params['classId']
        start_time_str = unquote(query_params['startTime'])
        end_time_str = unquote(query_params['endTime'])
    except (KeyError, TypeError):
        return {
            'statusCode': 400,
            'body': json.dumps({'error': 'Missing required query parameters:
classId, startTime, or endTime.'}),
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            }
        }

    sql_query = f"""
SELECT
    timestamp,
    currentCount,
    maximumCapacity
FROM
    "{athena_database}" "{athena_table}"
```

```

        try(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s'))
    ) <= COALESCE(
        try(date_parse('{end_time_str}', '%d/%m/%Y, %H:%i:%s.%f')),
        try(date_parse('{end_time_str}', '%d/%m/%Y, %H:%i:%s'))
    )
ORDER BY
    COALESCE(
        try(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s.%f')),
        try(date_parse(timestamp, '%d/%m/%Y, %H:%i:%s'))
    ) ASC;
"""

try:
    response = athena_client.start_query_execution(
        QueryString=sql_query,
        QueryExecutionContext={'Database': athena_database},
        ResultConfiguration={'OutputLocation': f's3://{s3_bucket}/query-
results/'})

    query_execution_id = response['QueryExecutionId']

    while True:
        query_status =
athena_client.get_query_execution(QueryExecutionId=query_execution_id)
        state = query_status['QueryExecution']['Status']['State']
        if state in ['SUCCEEDED', 'FAILED', 'CANCELLED']:
            break
        time.sleep(1)

    if state == 'FAILED':
        error_message =
query_status['QueryExecution']['Status']['StateChangeReason']
        print(f"Athena query failed: {error_message}")
        return {
            'statusCode': 500,
            'body': json.dumps({'error': f'Athena query failed:

```

```

        if 'ResultSet' in results_response and 'Rows' in
results_response['ResultSet']:
            for row in results_response['ResultSet']['Rows'][1:]:
                data_list = [d.get('VarCharValue') for d in row['Data']]
                if len(data_list) >= 3 and all(d is not None for d in
data_list):
                    try:
                        timestamp_str = data_list[0]
                        current_count = int(data_list[1])
                        maximum_capacity = int(data_list[2])
                        results.append({
                            'timestamp': timestamp_str,
                            'currentCount': current_count,
                            'maximumCapacity': maximum_capacity
                        })
                    except (ValueError, TypeError) as e:
                        print(f"Error parsing row: {data_list}. Error: {e}")

    return {
        'statusCode': 200,
        'body': json.dumps(results),
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        }
    }

except Exception as e:
    print(f"General error: {e}")
    return {
        'statusCode': 500,
        'body': json.dumps({'error': str(e)}),
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        }
    }

```

SendFacultyAlerts

```
import { SNSClient, PublishCommand } from "@aws-sdk/client-sns";
import { unmarshall } from "@aws-sdk/util-dynamodb";

const snsClient = new SNSClient({ region: "ap-southeast-1" });
const topicArn = "arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic";

export const handler = async (event) => {
  try {
    for (const record of event.Records) {
      const eventName = record.eventName; // INSERT, MODIFY, REMOVE
      const newImage = record.dynamodb.NewImage ?
unmarshall(record.dynamodb.NewImage) : null;

      if (!newImage) continue;

      // Extract values
      const classId = newImage.classId || newImage.classroomId || "Unknown";
      const fullCapacity = parseInt(newImage.maximumCapacity, 10) || 0;
      const currentCapacity = parseInt(newImage.currentCount, 10) || 0;
      const timestamp = newImage.timestamp || new Date().toISOString();

      // Only trigger alert if overloaded
      if (currentCapacity > fullCapacity) {
        const message = `Classroom ID: ${classId}
Full capacity: ${fullCapacity}
Current capacity: ${currentCapacity}
Status: The class is overloaded
Timestamp: ${timestamp}`;

        console.info("Publishing OVERLOAD message to SNS:", message);
```

```
    );  
  }  
}  
  
  return { statusCode: 200, body: "Processing complete." };  
} catch (err) {  
  console.error("Error publishing SNS message", err);  
  return { statusCode: 500, body: "Failed to send notification" };  
}  
};
```

FacultyAlertsTopic Access Policy

```
{
  "Version": "2008-10-17",
  "Id": "__default_policy_ID",
  "Statement": [
    {
      "Sid": "__default_statement_ID",
      "Effect": "Allow",
      "Principal": { "AWS": "*" },
      "Action": [ "SNS:Publish", "SNS:RemovePermission",
        "SNS:SetTopicAttributes", "SNS>DeleteTopic", "SNS:ListSubscriptionsByTopic",
        "SNS:GetTopicAttributes", "SNS:AddPermission", "SNS:Subscribe"
      ],
      "Resource": "arn:aws:sns:ap-southeast-1:019662059667:FacultyAlertsTopic",
      "Condition": {
        "StringEquals": {
          "AWS:SourceAccount": "019662059667"
        }
      }
    }
  ]
}
```